

The Map

Chapters 7–9: the data landscape, the career tree and how skills unlock roles, and how expertise actually forms.

Approved chapters · Version 1

Contents

Part I — The Map

Chapter 7. The Data Landscape

Chapter 8. The Career Tree: How Skills Unlock Roles

Chapter 9. How Expertise Actually Forms

Part I — The Map

Part I shows you the whole field before you start climbing it: who does which data work, how the roles connect, which skills open which doors, and how expertise is built over months and years. It's the part to reread whenever you're choosing what to learn next.

Chapter	What you'll be able to do	Time needed
7. The Data Landscape	place any data job under the four questions; describe the ten roles and three team structures; follow one request through every role	2–3 hours
8. The Career Tree: How Skills Unlock Roles	read the tiers and the skills matrix; decode a job description; understand pay sources; choose an entry route and plan backward from a role	3–4 hours
9. How Expertise Actually Forms	estimate your own timeline; practice deliberately; build portfolio pieces; get feedback; work through plateaus	2–3 hours

In total, allow 7–10 hours, including the exercises and projects. After Part I, Part II (Chapters 10–27) builds the analyst's toolkit, starting with spreadsheets.

Chapter 7. The Data Landscape

Chapter at a glance

You will learn to: name the four questions every data job answers, and place any job title under one of them · describe what each of the ten main data roles does, what it produces, and which tools it uses · explain why the field grows like a tree, with a shared trunk and branches that rejoin at the top · recognize the automation and integration track, and the automation hidden inside every other role · compare centralized, embedded, and hub-and-spoke data teams, and say when each fits · follow one business request through every role, from the first question to a running platform · judge what AI assistants change in each role, and what stays your responsibility.

Before you start: Chapter 1 (what data is) and Chapter 2 (files, databases, servers, and APIs). Chapter 3 (how a business runs on data) helps but isn't required.

Time needed: 2–3 hours, including the exercises and the project.

Tools: a pen and paper, or any free drawing tool for the project. No software to install.

Practice data: one short query on the Riverstone one-year database (`riverstone_2025`), shown so you can see what an analyst's answer looks like. You don't need to run it yet; Chapter 12 teaches every line. Every result and number shown is real.

Why this matters

From the outside, "working in data" sounds like a country where every city has the same name. Data analyst, business analyst, BI developer, analytics engineer, data scientist, ML engineer, data engineer, AI engineer, integration engineer, data architect: the titles blur together, and the job descriptions overlap. People new to the field often spend months learning tools for a job they turn out not to want, or apply for roles whose real work they don't understand.

The first skill in a data career isn't SQL or Python. It's **orientation**: knowing what the roles are, what each one produces, how they depend on each other, and where you stand among them. Once you have that, every later decision gets easier. You can read a job description and see the real job behind the title. You can pick what to learn next for a reason. And in an interview, you can explain where your work fits in the team, which interviewers notice.

This chapter is the map. Chapter 8 turns it into a route, skill by skill, and Chapter 9 is honest about how long the route takes.

In plain English

Think about what happens when you walk into a hospital with a pain in your side.

- At reception, someone asks what's wrong and writes it down in a way the doctors can use. Vague ("it hurts") becomes precise ("sharp pain, lower right, since yesterday"). That's the work of a **business analyst**: turning a vague request into a clear question.
- A doctor looks at your test results and says what's happening now. That's a **data analyst**: describing what happened, clearly and correctly.
- A specialist uses your history to judge what will probably happen next, and which treatment will work. That's a **data scientist**: prediction and cause.
- The lab technicians make sure every blood sample is collected, labeled, tested, and delivered to the right doctor on time. Nobody thanks them until a sample goes missing. That's a **data engineer**.
- The hospital's systems send your prescription to the pharmacy and a reminder to your phone, without anyone re-typing it. That's **automation and integration**.
- Someone designed the building: where the lab sits relative to the emergency ward, how records flow between departments, who is allowed to see what. That's a **data architect**.

One patient, one question, many specialists, all depending on each other. A data team works the same way. Keep this picture in mind; section 7.6 follows a single Riverstone request through every one of these roles.

7.1 The four questions every data job answers

Strip away the tools and the titles, and every data role exists to help answer one of four questions about an organization and its data.

1. **What happened?** Describing the past and present clearly: last month's sales, this week's late deliveries, which customers have stopped ordering, and the immediate reasons behind them.
2. **What will happen, and why?** Predicting what comes next, and finding out what causes what: which customers are likely to leave, how much stock to make for Diwali, whether a new discount really increased sales.
3. **How does data move, get served, and get put to work, reliably?** Building the machinery: collecting data from every system, checking it, storing it, and delivering it (as a report, an alert, or a task in another system) on time, every time, without someone doing it by hand.
4. **How should the whole system be designed?** Deciding how all the pieces fit together, who owns what, what it costs, and how it stays secure and trustworthy as the company grows.

Almost every data job is a specialization of one of these. When you meet an unfamiliar title, ask *"Which of the four questions does this person spend most of their day on?"* The title then stops being confusing.

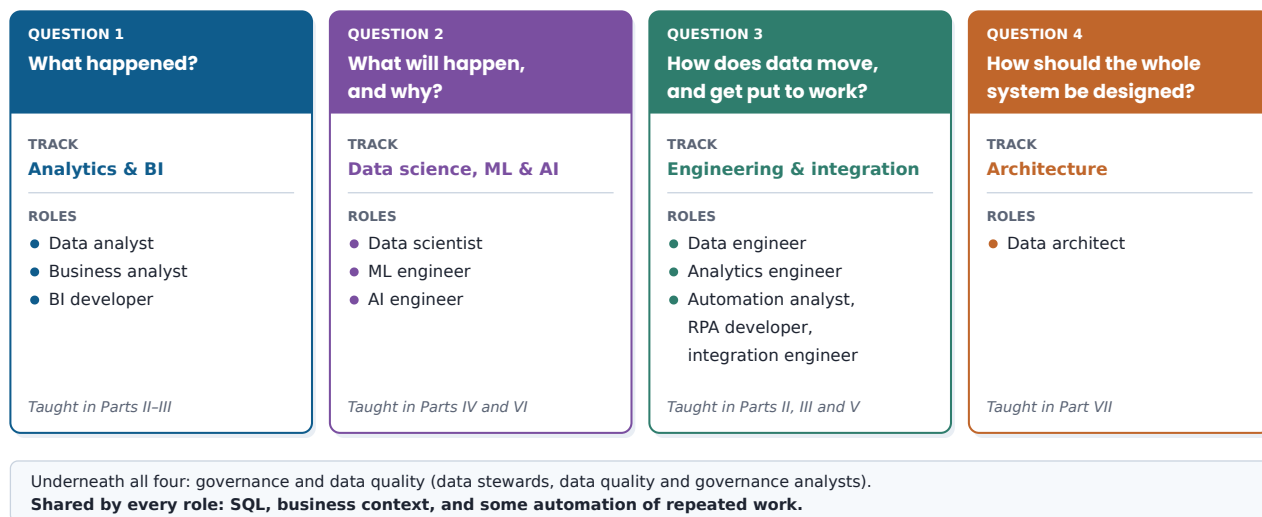


Figure 7.1 – The four questions, the tracks that answer them, and the ten roles this book follows. Notice the band at the bottom: some skills belong to everyone.

Simplification note. Real jobs cross these lines. Analysts ask "why?" all the time, and a good analyst explains *why* sales fell in March, not only *that* they fell. The difference is depth: an analyst finds the likely reason in the data; a data scientist designs an experiment or a model to prove a cause or predict an outcome. Use the four questions to find a role's center of gravity, not its edges.

7.2 The tracks and the ten roles

A **track** is a family of related roles that answer the same question. A **role** is a job with a recognizable set of responsibilities, whatever a particular company calls it. The same role might be advertised as "MIS executive", "reporting analyst", or "insights analyst", and the same title might mean different work at two companies. That's why this book describes roles by what they produce, not by title.

Here are the ten roles this book follows, grouped by track in the same order as Figure 7.1.

Track	Role	Center of gravity	What they typically produce	Core tools	Taught in
Analytics & BI	Data analyst	What happened, and why it probably happened	Answers to business questions, reports, analyses, recommendations	Excel or Google Sheets, SQL, a BI tool, some Python	Part II
Analytics & BI	Business analyst	What the business needs, and how a process should change	Clear requirements, process maps, user stories, test plans, the case for a change	Spreadsheets, SQL, process-mapping and ticketing tools	Ch 24-25

Track	Role	Center of gravity	What they typically produce	Core tools	Taught in
Analytics & BI	BI developer	Making "what happened" available to everyone, all the time	Dashboards, data models behind them, scheduled reports	Power BI or similar, SQL, DAX	Ch 16, 28
Data science, ML & AI	Data scientist	What will happen, and what causes what	Models, forecasts, experiment results, recommendations	Python, statistics, scikit-learn, SQL	Part IV
Data science, ML & AI	ML engineer	Keeping models working in the real world	Models running as services, with monitoring and retraining	Python, cloud services, MLOps tools	Ch 56
Data science, ML & AI	AI engineer	Building useful, safe products on top of AI models	Assistants, AI features, and AI steps inside business workflows, with evaluation	Python, LLM APIs, retrieval, evaluation tools	Ch 54–58
Engineering & integration	Data engineer	Getting data from every source, correct and on time	Pipelines, warehouses, data-quality checks	SQL, Python, orchestration tools, cloud	Part V
Engineering & integration	Analytics engineer	One trusted, tested version of the business's numbers	Clean, documented, tested tables that analysts and dashboards use	SQL, dbt, Git	Ch 32
Engineering & integration	Automation analyst, RPA developer, integration engineer	Removing manual steps, and connecting systems so data flows between them	Automated reports, low-code flows, bots, API integrations, data written back into business systems	Spreadsheet automation, Python, Power Automate or n8n, APIs	Ch 19–20, 51, 58
Architecture	Data architect	How the whole data system should be designed	Designs, standards, decision records, platform roadmaps	All of the above, at the level of design	Part VII

A few terms in that table will be new. A **dashboard** is a screen of charts and numbers that refreshes itself from the data. A **model**, in data science, is a set of rules learned from past data that makes predictions about new data. A **pipeline** is an automated series of steps that moves data from one place to another and changes it along the way. **BI** stands for business intelligence: the tools and practices for turning data into reports and dashboards people use to run the business. **MLOps** and **LLM** will each get full chapters; for now, MLOps is "keeping machine learning models healthy in production", and an LLM (large language model) is the kind of AI model behind chat assistants.

The one skill every role shares

Look down the *Core tools* column and one name keeps appearing: **SQL**. It's the language for asking questions of a database, and every role on this map writes it, often every day. The analyst uses it to answer questions. The BI developer uses it to feed dashboards. The analytics engineer builds whole

tested models of the business in it. The data scientist pulls training data with it. The data engineer moves and checks data with it. The integration engineer uses it to choose which records to push into the CRM. The architect reads it to understand what the system is really doing. That's why Part II teaches it at full depth in Chapters 12 and 13, and why nothing you learn there goes to waste, whichever branch you climb.

Watch out: titles lie, outputs don't. A "data scientist" at one company builds dashboards all day; a "business analyst" at another writes Python pipelines. Before you decide a job is (or isn't) for you, read what the person will *produce* each week. Chapter 8 shows how to decode a job description line by line.

Roles beyond the ten

The map has more cities than this book can visit in depth. **Data governance analysts** and **data stewards** make sure data is defined, documented, protected, and used lawfully. **Data quality analysts** hunt for and fix errors at the source. Specialist scientists work on text (NLP), images (computer vision), or recommendations. **Analytics managers**, **heads of data**, and **chief data officers** lead the people. Many domain roles combine a business function with data work: sales operations, finance analyst, supply chain analyst, marketing analyst. Each of these sits on the same map, and the chapters listed in "Where this leads" show where the book touches them.

7.3 The shape of the field: a trunk that branches

Here is the most important idea in this chapter: **the roles share a trunk and branch upward, and the branches rejoin at the top.**

Everyone starts with the same foundations: understanding data (Part 0), a spreadsheet, SQL, and business sense. The analyst core grows straight up from there (Part II), and many people build a whole career on it. Then the field branches. After the advanced analytics and analytics engineering skills of Part III, one branch leads toward data science and machine learning (Part IV), and the other toward data engineering and integration (Part V). The two branches meet again when models and AI have to run inside a real business (Part VI), and they come together fully at the top, in architecture (Part VII). A data architect can't design a system without having walked at least partway down each branch.

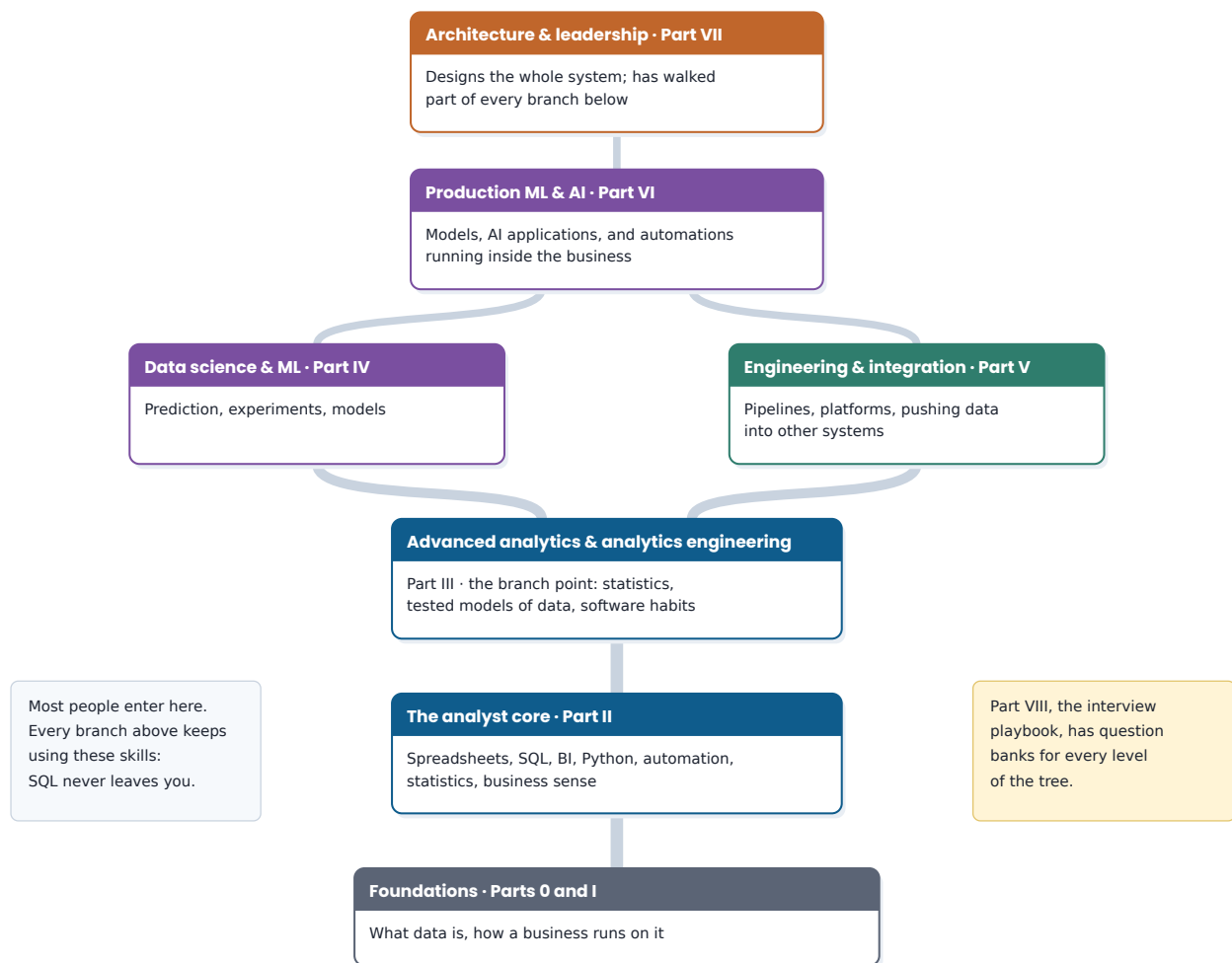


Figure 7.2 – The field grows like a tree, and this book follows it part by part. Read it from the bottom up: nobody starts at the top.

Two consequences follow, and they shape how you should use this book.

- **You don't abandon lower levels when you climb.** A data scientist still writes SQL every week, and an architect still reads spreadsheets. Skills accumulate.
- **Science and engineering are peers, not a ladder.** Neither branch is "above" the other. The book teaches data science (Part IV) before data engineering (Part V) because many analysts move toward science first, but plenty of excellent careers go straight from the analyst core to engineering. Chapter 8 turns this tree into tiers you can plan with, in the same order as the book's parts.

7.4 The automation and integration track

At every level of the tree, the same practical question comes up: *once the data is right, how does it reach the people and systems that act on it, without someone doing it by hand?*

In most companies, a large share of data work is exactly this. Someone exports a file from one system, pastes it into a spreadsheet, fixes the columns, adds formulas, saves a copy, and emails it to twenty managers. Next week they do it again. Chapter 2's "Friday file" was a small example of that kind of routine. Removing those steps is the job of the automation and integration track.

The track has three common roles, and the lines between them are blurry.

- An **automation analyst** finds manual, repeated work in business processes and automates it, often with spreadsheet automation (macros, VBA, Office Scripts, Google Apps Script), low-code tools such as Power Automate or n8n, and Python scripts. The role often grows out of an analyst or operations job.
- An **RPA developer** builds software "robots" that click through screens the way a person would. **RPA** stands for robotic process automation. It's the right tool when an old system has no other way in, and the wrong tool when a proper connection exists, because a robot breaks whenever the screen changes.
- An **integration engineer** connects systems directly, usually through APIs (Chapter 2), so data flows between them: new orders from the website into the ERP, payment status from finance into the CRM, a risk flag from the data warehouse back into the sales team's tools. Writing data from the warehouse back into business systems is called **reverse ETL** or **data activation**, and Chapter 51 teaches it.

Here's what makes this track different from the others: **every data role includes some automation**. The analyst schedules a refreshable report instead of rebuilding it. The BI developer sets up a dashboard subscription. The data engineer's whole job is automated pipelines. The ML engineer automates retraining. The architect decides which automations the company should build, own, and eventually retire. The automation roles specialize in it, but nobody in data is exempt.

Take a simple example. Riverstone's **Daily Sales Flash**, a one-page summary of yesterday's orders, could start as a manual report that takes someone 40 minutes each working day. If it's produced on 250 working days a year, that's $40 \times 250 = 10,000$ minutes, or about 167 hours: more than four working weeks spent re-typing the same report. Over the course of this book, that report becomes a SQL query (Chapter 12), a Python script (Chapter 18), a VBA macro and an Apps Script project (Chapter 19), a scheduled email in every manager's inbox (Chapter 20), a monitored pipeline (Chapters 46–47), a source of flags pushed into the CRM (Chapter 51), and finally part of a designed reporting platform (Chapter 63). The business need never changes. The way it's met gets better at each level, which is exactly how the job grows in real life.

AUTOMATED FLOW

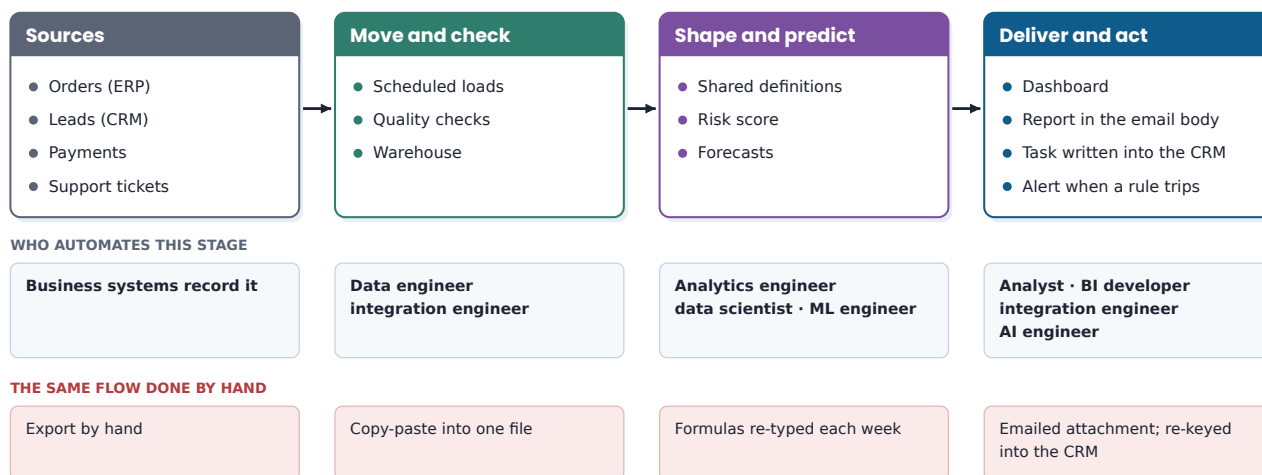


Figure 7.3 — The flow from source to action, with the roles that automate each stage. The red row is what the same flow looks like when people do it by hand.

Real-life example: the report nobody owns. Many companies run an important report on a macro written years ago by someone who has since left. It works until the source file's columns change, and then it fails quietly or, worse, produces wrong numbers that look right. Automating a report is only half the job. Someone has to own it, monitor it, and know what to do when it breaks. Chapter 63 covers that governance at the architect level.

7.5 How data teams are organized

Knowing the roles is half the map. The other half is how companies arrange those people, because it changes your day, your boss, and your career path. There are three common structures.

In a **centralized** team, all the data people sit in one team, usually under a head of data or a technology leader, and serve every department. Sales, finance, and operations send requests to that team.

In an **embedded** (or **decentralized**) setup, data people sit inside the departments they serve and report to those departments' heads: a sales analyst in the sales team, a finance analyst in finance. There's little or no central data team.

In a **hub-and-spoke** (sometimes called **federated**) structure, a central hub owns the shared platform, the standards, and the common definitions of key numbers, while analysts in each department (the spokes) do the day-to-day work. Spokes often report to their department but follow the hub's standards and meet with the hub regularly.

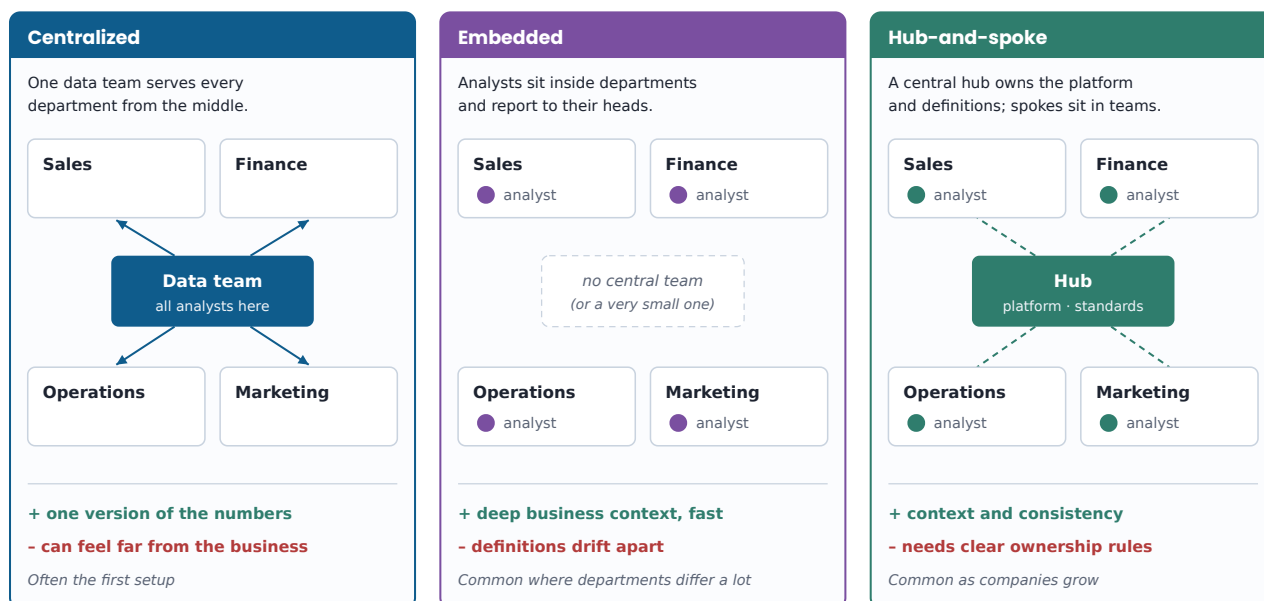


Figure 7.4 — Three ways to organize a data team. The same people can be arranged very differently, and each arrangement has a typical failure.

	Centralized	Embedded	Hub-and-spoke
Consistency of numbers	High: one team, one definition	Low: each department defines "revenue" its own way	High for shared numbers, if the hub enforces definitions
Understanding of the business	Can be weak; the team hears about problems secondhand	Strong; analysts sit in the meetings	Strong in the spokes
Speed for a department	Slower; requests wait in a queue	Fast	Fast for local work; shared changes take coordination
Duplicated work	Low	High: three teams build three versions of the same report	Medium
Career growth for data people	Clear; you learn from other data people	Harder; your manager may not understand your work	Clearer, through the hub's community and standards
Typical failure	The bottleneck: "the data team never gets to our request"	"Why do sales and finance report different revenue?"	Nobody is sure who owns a definition

No structure is right for every company. Small companies often start with one or two data people who behave like a tiny central team. As departments grow, analysts get hired into them, and inconsistent numbers start to hurt. Many companies then move toward hub-and-spoke to keep local speed while fixing definitions centrally. Chapter 66 goes deeper into building and structuring data teams.

Try it. Think of a company you know, or your college's administration. Where do the people who make reports sit? Which of the three structures is it closest to, and which typical failure have you heard people complain about?

7.6 One request, every role

Roles make the most sense in motion. So let's follow a single business request through all ten of them.

Simplification note. Riverstone is a mid-sized company, and in real life one or two people would cover most of these roles. To see each role clearly, imagine a larger version of Riverstone with one specialist in every seat. The data in step 2 is the real one-year database; the later steps describe what each role would build, and the chapters named teach you to build it.

It's the year-end review, and Anita Rao, the Sales Head, says:

"Some customers seem to have stopped ordering. Which ones, and what should we do about it?"

Step 1: The business analyst turns the request into a question

"Stopped ordering" can mean many things. The business analyst asks questions before anyone touches data: *Stopped since when? Does a cancelled order count? What about customers who signed up and never ordered at all? Who will act on the list, and what will they do? How often do you need it?*

The answers become a short requirement: **"List every customer with no non-cancelled order in the 60 days up to 31 December 2025, including customers who have never ordered. Sales reps will call them. Needed weekly."** That sentence took twenty minutes of conversation, and it will save days of rework.

Step 2: The data analyst answers it

The data analyst writes a query against the one-year database. You don't need to read SQL yet; look at the shape of the question in the code, then at the answer.

```
SELECT c.customer_name,
       c.segment,
       MAX(o.order_date)           AS last_order_date,
       DATE '2025-12-31' - MAX(o.order_date) AS days_since_last_order
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id
 AND o.status <> 'Cancelled'
GROUP BY c.customer_id, c.customer_name, c.segment
HAVING MAX(o.order_date) < DATE '2025-11-01'
      OR MAX(o.order_date) IS NULL
ORDER BY last_order_date NULLS FIRST;
```

customer_name	segment	last_order_date	days_since_last_order
Home Plus	Retail		
City Needs Store	Retail	2025-03-22	284
Sunrise Caterers	Hospitality	2025-06-10	204
Om Sai Provisions	Retail	2025-07-22	162
Tasty Tiffins	Hospitality	2025-10-26	66

(5 rows)

How it works, in plain words.

- The query starts from every customer, then looks up each one's orders, ignoring cancelled ones.
- For each customer, it finds the latest order date, and counts the days from that date to 31 December 2025.
- It keeps only customers whose latest order was before 1 November, plus customers with no orders at all.
- The blank dates are Home Plus: it signed up on 18 June 2025 and has never ordered, so there's no date to show.

Check one row by hand. Tasty Tiffins last ordered on 26 October 2025. From 26 to 31 October is 5 days, November has 30, and December has 31: $5 + 30 + 31 = 66$ days. ✓

A good analyst doesn't stop at the list. Looking closer, City Needs Store and Tasty Tiffins have placed only two orders each, so it's hard to call them "regulars". Sunrise Caterers and Om Sai Provisions placed four orders each, roughly every five to six weeks, and then went silent. Together those two brought in ₹114,072.50 of revenue in 2025, about 2.6% of Riverstone's ₹4,335,471 for the year.

What to tell Anita. Five customers haven't ordered in 60 days. Two of them, Sunrise Caterers and Om Sai Provisions, used to order regularly, so they're the most urgent calls: find out what went wrong. Home Plus signed up in June and never ordered: a lead that was never converted. The other two ordered only twice, so they may be occasional buyers.

SQL link. Chapter 12 teaches every clause in this query and includes a version of it on the mini database (section 12.15). Chapter 13 improves the rule itself, comparing each customer's silence with their own usual ordering rhythm (Pattern 6).

Dialect note. The same question in MySQL needs two changes: `DATEDIFF` instead of subtracting dates, and a manual sort to put the blank dates first, because MySQL has no `NULLS FIRST`. The result is the same five customers.

```
SELECT c.customer_name,
       c.segment,
       MAX(o.order_date)                AS last_order_date,
       DATEDIFF('2025-12-31', MAX(o.order_date)) AS days_since_last_order
FROM customers AS c
LEFT JOIN orders AS o
      ON c.customer_id = o.customer_id
      AND o.status <> 'Cancelled'
GROUP BY c.customer_id, c.customer_name, c.segment
HAVING MAX(o.order_date) < '2025-11-01'
      OR MAX(o.order_date) IS NULL
ORDER BY last_order_date IS NOT NULL, last_order_date;
```

customer_name	segment	last_order_date	days_since_last_order
Home Plus	Retail	NULL	NULL
City Needs Store	Retail	2025-03-22	284
Sunrise Caterers	Hospitality	2025-06-10	204
Om Sai Provisions	Retail	2025-07-22	162
Tasty Tiffins	Hospitality	2025-10-26	66

Step 3: The BI developer makes it available every day

Anita likes the answer and wants it every week without asking. The BI developer adds an "At-risk customers" page to the sales dashboard. It refreshes from the database each morning, each rep can filter it to their own customers, and Anita gets it by email every Monday through a dashboard subscription. The analyst no longer has to re-run the query by hand.

Step 4: The analytics engineer makes the definition trustworthy

A month later, finance mentions that *their* "inactive customers" report uses 90 days, not 60. The customer-support team uses a third rule. Three reports, three answers, and every meeting starts with an argument about whose number is right.

The analytics engineer agrees one definition with sales and finance, writes it once as a tested, documented table (a `customer_activity` model, in the style of Chapter 32), and points every report at it. Tests run automatically: no customer can appear twice, every customer must have a status, and the total number of customers must match the source system. "Active customer" now means one thing everywhere.

Step 5: The data scientist asks what will happen next

The list tells Riverstone who *has* gone quiet. The data scientist asks a better question: *can we tell who is **about to** go quiet, while there's still time to help?* They look for early warning signs in the history: orders getting smaller, gaps getting longer, late payments, complaints to support. They build a model that gives each customer a risk score, and test it fairly against past data: does it beat the simple 60-day rule, or is it an expensive way to get the same list? If it does beat it, they suggest a fair test: call half the high-risk customers, leave the other half alone for a month, and compare what happens. That test is how you learn whether the calls work.

Step 6: The ML engineer keeps the model running

A model in a notebook helps nobody on Monday morning. The ML engineer turns it into a service that scores every customer each night, logs every score, and raises an alarm if the model's accuracy starts to slip, for example if customer behavior changes after a price increase. They also set up a way to retrain it safely. Chapter 56 covers this work.

Step 7: The data engineer makes sure the data arrives

The model needs more than orders. It needs payments from finance, tickets from the support desk, and sales activity from the CRM, all fresh by 6 a.m. The data engineer builds pipelines that pull data from each system every night, check it (did yesterday's orders arrive? are there duplicate payments?), and load it into the warehouse. If a check fails, the pipeline stops and alerts someone *before* a wrong risk score reaches a sales rep.

Step 8: The automation or integration engineer puts it to work

A score sitting in a warehouse changes nothing. The integration engineer writes the high-risk customers into the CRM as call tasks for the right rep, making sure the same customer doesn't get a duplicate task every night. They also set up a Monday email to each rep with their own list in the body of the email, not as an attachment nobody opens. If the old finance system had no API, this is where an RPA bot might be considered, as a last resort.

Step 9: The AI engineer helps the rep prepare

Before calling Sunrise Caterers, a rep wants to know the story: last orders, open invoices, recent complaints. The AI engineer builds an assistant that drafts a short call brief from Riverstone's own data. The design matters more than the demo: the assistant may only use facts it retrieved from the company's systems, it shows where each fact came from, the rep reads and approves the brief before using it, and the team tests it regularly for invented facts. Chapters 55 and 58 teach this.

Step 10: The data architect designs how it all fits

Stand back and look at everything that now exists: pipelines, a warehouse, a shared definition, a model service, a dashboard, CRM tasks, emails, an AI assistant. The data architect decides how these pieces should fit together, so the company doesn't end up with a tangle of scripts nobody understands. Where does the risk score officially live? Who owns the definition of "active"? Should reps see only their own customers? What happens if the CRM write-back fails halfway? What does the whole thing cost each month, and is it worth it? The architect writes these decisions down, with the reasons, so the next person can understand them.

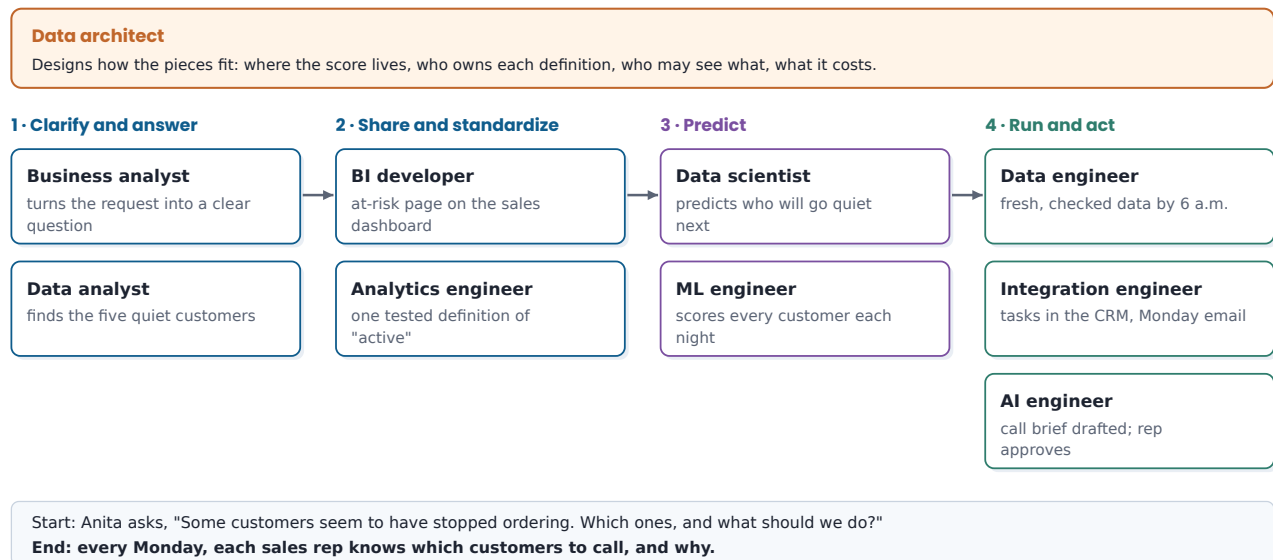


Figure 7.5 – One request, every role. Notice that the order matters: prediction is worth building only after the question is clear, the data is trusted, and the definition is agreed.

What the walk-through shows

Three things stand out.

- **Most of the value came early.** Steps 1 and 2 answered Anita's question in an afternoon. Everything after that makes the answer faster, more consistent, more predictive, or more automatic. Those are real improvements, but a company should build them only when the simpler version is in use and its limits are hurting.
- **Every role depends on the ones before it.** A model built on an unclear question, or on data nobody checks, produces confident nonsense.
- **Automation appears at almost every step.** The subscription, the tests, the nightly scoring, the pipeline, the CRM tasks, the email: each one removes a manual step someone would otherwise repeat forever.

7.7 How AI assistants are changing each role

AI assistants can now draft formulas, SQL, Python, documentation, and summaries in seconds. It's natural to wonder whether that makes these roles, or the skills in this book, less needed. The pattern so far is more useful than either hype or fear.

Adoption is high, and trust is limited. In Stack Overflow's 2025 Developer Survey, a large survey of people who write code, 84% of respondents said they use or plan to use AI tools in their work, up from 76% the year before. In the same survey, more respondents said they distrust the accuracy of AI output (46%) than trust it (33%). The survey covers developers generally, not data roles specifically, but the message fits data work well: people use these tools constantly, and they check what the tools produce.

Demand for data skills is still growing. The World Economic Forum's *Future of Jobs Report 2025*, based on a survey of employers, lists big data specialists and AI and machine learning specialists among the fastest-growing jobs to 2030, and data entry clerks among the fastest-declining. Employers in the same report rate analytical thinking as the most sought-after core skill. Read the direction, not the exact rankings: the work that is shrinking is re-typing data; the work that is growing is understanding and using it.

Here's how that plays out role by role.

Role	What AI assistants speed up	What stays your responsibility
Data analyst	First drafts of formulas and queries; explaining an unfamiliar table; summarizing results	Knowing which question matters; checking the numbers; telling the manager what to do
Business analyst	Meeting notes, first drafts of requirements and user stories	Noticing what stakeholders didn't say; resolving disagreements; deciding what's in scope
BI developer	Suggesting charts; drafting measures	Choosing what the dashboard should help people decide; making sure the numbers match the source
Analytics engineer	Drafting tests and documentation	Getting departments to agree on one definition
Data scientist	Boilerplate code; lists of candidate features	Framing the problem; spotting data leakage; designing fair tests; saying "the model isn't worth it"
ML engineer	Deployment and monitoring scripts	Deciding what to monitor, and what to do when a model degrades
Data engineer	Pipeline code; translating SQL between databases	Designing for failure; agreeing data contracts with source-system owners
Automation / integration	Building flows from a plain-language description	Handling exceptions, permissions, and what happens when a step fails silently
AI engineer	Much of the code itself	Evaluating quality and safety; deciding where a human must approve
Data architect	Drafting design documents; comparing options	Making trade-offs and being accountable for them

The right column is the durable part of each job. Notice what those tasks have in common: they need context about the business, judgment about trade-offs, and someone who is accountable when things go wrong. The left column holds tasks with a checkable right answer, and those are what assistants do well, *if* someone who understands the work checks them.

That leads to the rule this book follows throughout: **you are responsible for every number you deliver, whoever or whatever typed the formula.** An assistant that writes a query you can't read hasn't saved you time; it has moved the risk to the moment your manager acts on a wrong number. So the book teaches you to read, write, and check the work yourself, then shows how to use assistants to go faster. Chapter 6 covers learning with AI assistants without letting them think for you, and Chapter 26 covers using them well at work.

Watch out: "almost right" is the dangerous kind of wrong. An AI-drafted query that fails with an error is harmless: you notice. One that runs and returns a plausible total, while quietly counting cancelled orders, is the one that reaches the board. Hand-check one row and reconcile to a known total, every time.

7.8 Where you are right now

Wherever you're reading from (a spreadsheet job, an operations role, a college course, or a complete standing start), you're somewhere on this map. Maybe you already answer Question 1 in Excel every week without calling it analysis. Maybe you're the person who automated a painful report and became the office's unofficial automation analyst.

Locate yourself truthfully, and you'll see that the next step is usually closer than it looks. Chapter 8 gives you the tool to plan it: a career tree that shows which skills unlock which roles, what each role's day looks like, how to read a job description, and which parts of this book to read for your goal.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Choosing a target role by title or by what sounds impressive	You're learning deep learning, but the jobs you apply for want SQL and dashboards	Pick a role by the question it answers and what it produces each week (sections 7.1-7.2)
Assuming a job title means the same thing everywhere	Two "data analyst" postings ask for completely different skills	Read the responsibilities and outputs, not the title; Chapter 8 shows how
Thinking data science is "the senior version" of analysis	You feel you must leave analysis to progress	Senior analysts are highly valued; science and engineering are branches, not promotions (section 7.3)
Skipping the analyst core to reach a "higher" branch	Models and pipelines built on questions you can't frame or data you can't check	Build the trunk first: spreadsheets, SQL, business sense
Treating automation as someone else's job	The same report takes you hours every week	Every role automates; start with refreshable reports and scheduled delivery (section 7.4)
Automating a report without owning it	It fails silently when a source file changes, and nobody notices for weeks	Assign an owner, add a check, and write down what to do when it breaks
Jumping to prediction before the basics work	A churn model nobody uses, while "active customer" still has three definitions	Clarify, answer, share, and standardize before you predict (section 7.6)
Believing one team structure is always best	Endless reorganizations, same complaints	Match the structure to the company's size and pain; know each structure's typical failure (section 7.5)

Mistake	Symptom	Fix
Assuming AI assistants make core skills unnecessary	You can't tell when a drafted query is wrong	Learn to read and check the work; use assistants to go faster, not to replace understanding
Pasting AI output into a report unchecked	A plausible number that doesn't reconcile to any known total	Hand-check a row and reconcile to a total before anything leaves your hands
Picking RPA because it's quick to demo	A bot that breaks whenever a screen changes	Use an API or a direct integration where one exists; RPA only as a last resort
Waiting to "finish learning" before locating yourself	Months of study with no clear target	Place yourself on the map now; it will change, and that's fine

In the real world: Anita wants to hire a data scientist

Anita Rao came back from an industry conference with a clear idea. "Everyone is doing AI," she told Meera Iyer on Monday morning. "We need a data scientist. Someone who can predict which customers we're about to lose. Can you help me write the job advertisement?"

Meera had been at Riverstone long enough to inherit Imran's Friday file, learn SQL, and become the person everyone asked for numbers. She didn't want to say no to her boss. She also didn't think a data scientist was what Riverstone needed first.

"Before we write it," she said, "can I show you what the job would involve day to day? It'll take fifteen minutes."

She drew the four questions on the whiteboard and asked Anita which one she needed answered. Anita pointed at the second: *what will happen?* Then Meera opened the year's data and ran the query from her customer review. Five customers had gone 60 days without an order. Two of them, Sunrise Caterers and Om Sai Provisions, used to order every five or six weeks and had been silent for months. Nobody in sales had noticed.

"We didn't need a prediction to catch those two," Meera said. "We needed someone to look, every week. A model would have flagged them earlier, maybe. But right now the list isn't even on anyone's desk."

Anita frowned. "So you're saying we don't need a data scientist."

"I'm saying we don't need one yet," Meera said. She walked through what a data scientist would need to be useful: a clear definition of a lost customer, which Riverstone didn't have (sales and finance used different rules); clean history joined across orders, payments, and support complaints, which currently lived in three places, one of them an Excel file; and somebody to act on the scores every week. "Without those, a data scientist would spend their first six months doing my job and a data engineer's job, and they'd probably leave, because that's not the work they were hired for."

Then she listed the manual work she did every week: 40 minutes each morning on the Daily Sales Flash, most of a Friday on the sales pack, and copying overdue invoices from the finance system into emails for reps. "If we hire an analyst who can automate, those hours come back in the first three

months. That person sets up the at-risk list as a Monday email to every rep, fixes the definition of 'active customer' with finance, and gets the data into one place. Once that's running and we have a couple of years of clean history, a data scientist has something to work with."

Anita was quiet for a moment. "The conference speaker made it sound like step one."

"For a company with millions of customers and a data platform, it might be," Meera said. "For us, it's step five."

They rewrote the advertisement together. The title became *Data Analyst (Sales Analytics and Automation)*. The responsibilities listed the Monday at-risk list, a sales dashboard, agreed definitions with finance, and automating recurring reports. The skills section asked for SQL, Excel or Google Sheets, a BI tool, and "experience automating a recurring report, with any tool". Under *Nice to have*, Meera added "interest in forecasting and customer analytics", so the new hire could grow toward the role Anita had first imagined.

What made this work.

- **Meera answered the real need, not the requested title.** Anita wanted to stop losing customers. A title was her guess at the solution.
 - **She used the map.** Framing the conversation around the four questions and the order of roles from section 7.6 turned an opinion ("I don't think we need that") into a plan.
 - **She showed evidence from the data.** Two lapsed regular customers nobody noticed made the case better than any argument.
 - **She didn't kill the ambition.** The data-science path stayed in the plan, in the right place.
-

Tools

This chapter needs no software. For the project you'll need:

- **Pen and paper**, or a free diagram tool such as **diagrams.net** (also called draw.io) or **Excalidraw**, to draw your team map and a request's journey.
 - **A few job postings** from any job portal, for exercise 2 and the stretch goal. Save or print them; postings disappear.
 - **Companion file (optional):** `companion/mysql/ch07_queries_mysql.sql` holds the section 7.6 query in PostgreSQL and MySQL form, for readers who already have the practice databases loaded (Chapter 12, section 12.3, explains the setup).
-

The project: map the data work around you

Goal: a one-page map that shows who does which data work in an organization, how a request travels, and where manual work hides. It's a small portfolio piece, and it's excellent preparation for "tell me about how your team works" in an interview.

Option A: your own workplace or college. Use a real team you know.

Privacy reminder. Don't put colleagues' names, salaries, or confidential systems in anything you share publicly. Use role names ("sales coordinator", "finance team") instead of people's names, and leave out anything your employer wouldn't want outside the building.

Option B: Riverstone Supplies. Use the people and systems you've met in Chapters 1, 2, and this chapter: Anita, Vikram, the sales executives, Meera, the Friday file, and the Daily Sales Flash.

Steps

1. **List the recurring data requests.** Write down at least five reports, lists, or numbers people regularly ask for. For each, note who asks, how often, and in what form it arrives (a file, an email, a dashboard, a phone call).
2. **Tag each request with one of the four questions.** Most will be Question 1. That's normal.
3. **Name the roles doing the work today,** whatever their titles. Where one person covers several roles, say so ("Meera: analyst, BI developer, and automation analyst").
4. **Draw the team structure.** Is it centralized, embedded, hub-and-spoke, or "one person doing everything"? Sketch it in the style of Figure 7.4.
5. **Trace one request from start to finish,** in the style of Figure 7.5: who asks, who clarifies, who pulls data, who checks it, how it's delivered, who acts on it.
6. **Mark every manual step in red:** exports, copy-paste, re-typing, emailed attachments. Estimate the minutes each one takes, and multiply by how often it happens in a year.
7. **Write three sentences:** the biggest risk in this flow, the single automation that would save the most time, and which role you'd hire (or become) next, with one reason.

Stretch goals

- Add where AI assistants are used today (or could be), and what someone checks before the output is used.
- Collect three job postings for the role you picked in step 7 and underline what each would produce in its first three months.
- Redraw the map for the organization as it might look in three years, and write down which structure you'd recommend and why.

You've got it when...

- ☐ You can name the four questions and place any data job title under one of them.
- ☐ You can describe what each of the ten roles produces, in one sentence each, without using its title.
- ☐ You can explain why SQL is useful in every role on the map.
- ☐ You can sketch the tree in Figure 7.2 from memory and explain why science and engineering are peers.

- ☐ You can explain the difference between an automation analyst, an RPA developer, and an integration engineer, and when RPA is the wrong choice.
 - ☐ You can give one example of automation inside a role that isn't an automation role.
 - ☐ You can compare centralized, embedded, and hub-and-spoke teams, including each one's typical failure.
 - ☐ You can follow a business request through the roles in a sensible order, and explain why prediction comes after a clear question and trusted data.
 - ☐ You can say, for your target role, what AI assistants speed up and what stays your responsibility.
 - ☐ You've placed yourself on the map, and can say which role you're aiming for next.
-

Recap

- **Orientation** comes before tools: know what each role does before choosing what to learn.
 - Every data job answers one of **four questions**: what happened; what will happen, and why; how data moves, gets served, and gets put to work; and how the whole system should be designed.
 - The book follows **ten roles**: data analyst, business analyst, BI developer, analytics engineer, data scientist, ML engineer, data engineer, AI engineer, automation analyst / RPA developer / integration engineer, and data architect. Judge roles by what they produce, not by title.
 - **SQL** is the one skill every role on the map uses.
 - The field grows like a **tree**: a shared trunk (foundations and the analyst core), a branch point (Part III), two peer branches (science and engineering), and a top where they rejoin (production AI and architecture).
 - The **automation and integration track** removes manual steps and connects systems, using spreadsheet automation, low-code flows, **RPA**, APIs, and **reverse ETL**. Every data role includes some automation.
 - Data teams are **centralized, embedded, or hub-and-spoke**; each trades consistency, business context, and speed differently, and each has a typical failure.
 - Following one request through every role shows the right order: clarify, answer, share, standardize, predict, then run and act, with the architect designing the whole.
 - **AI assistants** speed up tasks with a checkable answer; context, judgment, and accountability stay with you. You're responsible for every number you deliver.
-

Practice exercises

Warm-up

1. For each request, name which of the four questions it mainly answers. (a) "How many orders did we ship late last month?" (b) "How much stock of Storage Box 25L should we make for October?" (c) "Can the website's new orders reach the ERP without anyone re-typing them?" (d) "Should the

- warehouse, the CRM, and the finance system share one customer ID, and who should own it?" (e) "Did the 5% hospitality discount actually increase orders, or would they have grown anyway?"
2. Which role from section 7.2 is each person closest to, whatever their title? (a) Priya spends her week building tested SQL tables that define revenue, margin, and active customers for every dashboard in the company. (b) Arjun connects the e-commerce platform to the ERP through APIs and makes sure no order is loaded twice. (c) Sana meets department heads, maps how purchase approvals work today, and writes the requirements for a new approval system. (d) Karthik keeps a demand-forecasting model running, monitors its accuracy, and retrains it when it drifts.
 3. A company's finance and sales teams each have their own analysts, who report to their department heads. There's no central data team. Which structure is this, and what problem would you expect to hear about in meetings?

Core

4. In the section 7.6 result, Home Plus has blank `last_order_date` and `days_since_last_order` values. What do the blanks mean, and why would it be a mistake to leave Home Plus off Anita's list?
5. Check by hand that City Needs Store's `days_since_last_order` of 284 is correct. Its last order was on 22 March 2025, and "today" is 31 December 2025.
6. Anita can make two calls this afternoon. From the five customers in section 7.6, which two should she call, and why? Use the order counts given in the analyst's notes.
7. Meera spends 25 minutes each working day copying overdue invoices from the finance system into emails for sales reps. Assuming 250 working days a year, how many hours is that per year? About how many 40-hour working weeks? Which role's skills would remove this work, and name one way they might do it.
8. Riverstone has grown. It now has two analysts in sales, one in finance, and one in operations, all reporting to their department heads. In the monthly review, sales and finance report different revenue totals for the same month. Recommend a team structure, and describe the first two things the new setup should do.
9. Rewrite this vague request as a precise question a data analyst could answer, in the style of section 7.6, step 1: "Are our hotel customers happy?" List at least three clarifying questions you'd ask first.
10. For each automation, say whether an API integration or an RPA bot is the better choice, and why.
(a) Copying new website orders into the ERP, which has a documented API. (b) Downloading a monthly statement from an old supplier portal that only offers a login page and a "Download" button, with no API.

Stretch

11. Finance says "inactive" should mean *no non-cancelled order in 90 days, including customers who never ordered*. Using only the section 7.6 result, how many customers would be on finance's list, and which one drops off? Which role would fix the problem of two definitions, and how?
12. An AI assistant drafts a query that returns *four* quiet customers under the 60-day rule, not five. Without running anything, give three likely reasons for the difference and say how you'd check each one.

- l3. Trace a new request through the roles: "Our deliveries to Chennai keep arriving late. Why, and can we stop it?" Name the roles you'd involve, in order, what each produces, and which roles you'd leave out for now, with reasons.

Think about it (no query needed)

- l4. Why does a data architect need to have walked at least partway down both the science and the engineering branches? Give one concrete decision from section 7.6, step 10, that would go wrong without that experience.
- l5. Some people argue that automation shouldn't be a separate track, because every role automates. Others argue it deserves its own roles. Make the best case for each view in two or three sentences, then say which fits a company like Riverstone today.

Key terms

orientation · track · role · data analyst · business analyst · BI developer · analytics engineer · data scientist · ML engineer · data engineer · AI engineer · automation analyst · RPA developer · integration engineer · data architect · dashboard · model · pipeline · BI (business intelligence) · MLOps · LLM (large language model) · SQL · data governance analyst · data steward · data quality analyst · RPA (robotic process automation) · reverse ETL (data activation) · Daily Sales Flash · centralized team · embedded (decentralized) team · hub-and-spoke (federated) team

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 8, The Career Tree: How Skills Unlock Roles**, turns this map into tiers in the book's part order, with a skills matrix per role, a day in the life of each role, how to decode a job description, the reader pathways table, and entry routes for freshers, career switchers, and internal moves.
- **Chapter 9, How Expertise Actually Forms**, is honest about the timeline, and shows how to practice, build a portfolio, find feedback, and get through plateaus.
- **Chapters 12 and 13** teach the SQL behind section 7.6's query, and the better "customer rhythm" version of the at-risk list.
- **Chapters 19 and 20** start the automation thread in earnest: spreadsheet automation, and reports delivered by email and on schedule. **Chapter 25** covers the business analyst track, including finding automation opportunities.
- **Chapters 51, 58, and 63** take automation to the engineering, AI, and architect levels. **Chapter 66** returns to building and structuring data teams.
- **Part VIII, Chapter 68, How Data Hiring Works**, explains the interview rounds for each of these roles. Every role in this chapter has matching question banks in Chapters 70–82.

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) Question 1, what happened: it describes last month. (b) Question 2, what will happen: it's a forecast. (c) Question 3, how data moves and gets put to work: it's an integration between two systems. (d) Question 4, how the whole system should be designed: shared IDs and ownership across systems are architecture decisions. e. Question 2, *why*: it asks whether the discount *caused* the increase, which needs an experiment or a careful causal analysis. The common wrong answer is Question 1, because it sounds like "what happened to orders". Describing that orders rose is Question 1; proving the discount caused it is Question 2.

2. (a) Priya is an **analytics engineer**: tested, shared definitions used by every report. (b) Arjun is an **integration engineer**: API connections and duplicate-safe loading. (c) Sana is a **business analyst**: process mapping and requirements. (d) Karthik is an **ML engineer**: keeping a model running, monitored, and retrained. Titles at their companies might be anything; the outputs tell you the role.

3. It's an **embedded** (decentralized) structure. Expect the typical failure from the table in section 7.5: the departments' numbers disagree ("why does finance's revenue differ from sales?"), and similar reports get built more than once.

4. The blanks are NULLs: Home Plus has no non-cancelled orders at all, so there's no latest order date and nothing to count days from. Leaving it off would be a mistake because a customer who signed up and never ordered is the easiest kind to miss and often the easiest to win: someone showed interest and nobody followed up. That's why the business analyst's requirement in step 1 explicitly included never-ordered customers, and why the query keeps rows where the latest order date is missing.

5. From 22 March to 31 March is 9 days. April 30, May 31, June 30, July 31, August 31, September 30, October 31, November 30, December 31. Total: $9 + 30 + 31 + 30 + 31 + 31 + 30 + 31 + 30 + 31 = 284$ days.
✓ The common slip is counting 22 March itself, which gives 285; days *since* the last order start counting the day after it.

6. **Sunrise Caterers and Om Sai Provisions**. Each placed four orders at a regular rhythm (roughly every five to six weeks) and then went silent for 204 and 162 days, so something has probably gone wrong, and they were worth ₹114,072.50 together in 2025. City Needs Store and Tasty Tiffins ordered only twice each, so their silence may be their normal pattern. Home Plus is worth a call too, but as a sales lead to convert rather than a customer to win back, so it can go to a sales executive. A reasonable alternative answer puts Home Plus second, if Anita's priority is new business; what matters is giving a reason.

7. $25 \times 250 = 6,250$ minutes per year. $6,250 \div 60 \approx 104.2$ hours, and $104.2 \div 40 \approx 2.6$ working weeks. This is **automation analyst** (or integration engineer) work, and an analyst who automates could do it too. Possible approaches: a query or saved report that lists overdue invoices by rep, delivered as a scheduled email with each rep's list in the email body (Chapter 20); or a low-code flow in Power Automate or n8n that runs every morning (Chapter 20); or, at a later stage, overdue flags pushed straight into the CRM as tasks (Chapter 51).

8. A **hub-and-spoke** structure fits: the analysts stay close to their departments, but a small hub (even one person to begin with) owns shared definitions and the platform. First two things: (1) agree one written definition of revenue with sales and finance (for example, net of discount, excluding cancelled orders, by order date or by invoice date, but one of them), and build it once as a shared table; (2) point both departments' reports at that shared table and reconcile the old numbers so everyone understands why they differed. Moving everyone into a fully centralized team would also fix consistency, but it would lose the business context the embedded analysts already have.
9. Clarifying questions could include: *What does "happy" mean to you: ordering more, complaining less, paying on time, or a survey score? Which customers count as "hotel": the Hospitality segment, which also includes caterers? Over what period? Compared with what: last year, or other segments? Who will act on the answer, and what decision is it for?* One precise version: **"For Hospitality customers, compare the number of orders, revenue, and support complaints per customer in the last six months of 2025 with the six months before, and list customers whose orders fell by more than half."** Any precise version is acceptable if it names the group, the measures, the period, and the comparison.
10. (a) **API integration.** A documented API is designed for systems to exchange data; it's faster, and it doesn't break when a screen's layout changes. (b) **RPA bot** is reasonable here, because there's no other way in. Treat it as fragile: monitor it, and ask the supplier whether a data feed or an emailed statement is available, so the bot can be retired.
11. Finance's rule keeps customers with more than 90 days since their last order, plus never-ordered customers: Home Plus (never), City Needs Store (284), Sunrise Caterers (204), and Om Sai Provisions (162). That's **4 customers. Tasty Tiffins drops off**, at 66 days. An **analytics engineer** would fix the problem of two definitions: agree one definition (or two clearly named ones, such as "quiet 60" for sales follow-up and "inactive 90" for finance), build it once as a tested, documented table, and point every report at it. The wrong fix is to let each team keep its own hidden rule and argue in meetings.
12. Likely reasons: (1) **Never-ordered customers were dropped.** An inner join, or a filter written as a plain `WHERE` condition on orders, removes Home Plus. Check: does Home Plus appear? (2) **Cancelled orders were counted as activity.** A customer whose only recent order was cancelled would look active. Check: look for cancelled orders after 1 November for the missing customer. (3) **A different cut-off date or comparison.** For example `<=` instead of `<`, a cut-off of 60 days from a different "today", or `CURRENT_DATE` instead of 31 December 2025. Check: compare the dates in the query with the requirement. In every case, reconcile: list the customers in both results and investigate the one that differs. Here, reason (1) is the most likely, because Home Plus is exactly the kind of row that inner joins silently lose.
13. A sensible order: **Business analyst** first, to define "late" (against the promised date? by how many days?) and find who needs the answer. **Data analyst** next, to measure how often Chennai deliveries are late, compared with other cities, and by delivery partner, route, product, or day of the week, and to suggest the likely causes. **BI developer**, if it needs watching every week, to add an on-time delivery page to the operations dashboard. **Automation or integration engineer**, if delivery status has to be copied from the delivery partners' systems by hand, to connect those systems. Leave out the **data scientist** and **ML engineer** for now: prediction is premature until the cause is understood and the

data is reliable; they might come in later to predict late deliveries before dispatch. The **data architect** isn't needed for one analysis, though they'd care if delivery data turned out to be missing from the platform. Other reasonable orders are fine if each step depends on the one before.

14. Because the architect's decisions sit where the branches meet, and each branch fails in different ways. Example from step 10: *"What happens if the CRM write-back fails halfway?"* Without engineering experience, an architect might not know that retries can create duplicate tasks unless each write can be safely repeated. Another example: *"Where does the risk score officially live?"* Without science experience, they might not realize the score needs to be stored with the model version and date that produced it, or nobody can later explain why a customer was flagged. An architect who has never met these problems tends to design systems that look clean on paper and break in production.

15. *For a separate track:* automation at scale needs specialist knowledge of APIs, integration platforms, RPA, error handling, and governance, and companies with many systems benefit from people who do it full time and own the automations. *Against:* the best automations come from people who understand the work being automated, so every role should automate its own repeated tasks, and a separate team can become a bottleneck. For a company like Riverstone today, most automation should come from analysts who automate their own reports (as in the story), with a specialist integration role added when many systems need connecting. Any answer that weighs both sides and ties the choice to the company's size is acceptable.

Chapter 8. The Career Tree: How Skills Unlock Roles

Chapter at a glance

You will learn to: use the "keys and doors" idea to decide what to learn next · name the seven tiers of the career tree and the roles each one unlocks · read a skills matrix to see what each role really needs · picture a normal working day in each of the ten roles · decode a job description line by line and score your own fit · read salary figures carefully, and check them yourself · choose a reading pathway through this book for your goal · plan an entry route as a fresher, a career switcher, or through an internal move.

Before you start: Chapter 7 (the four questions, the ten roles, and how teams are organized).

Time needed: 3–4 hours, including the exercises. Allow another 2–3 hours for the project, which uses real job postings.

Tools: a spreadsheet (Excel or Google Sheets) or paper, and a job portal for the project. Nothing to install.

Practice data: a fictional job description from Riverstone Supplies, the skills matrix in section 8.3, and salary figures published by PayScale and Indeed for India, as retrieved in September 2026. Every calculation shown is checked.

Why this matters

"What should I learn next?" is the most common question in every data community, and it's usually asked backwards. People ask about tools in isolation: *Should I learn Spark? Is Tableau worth it? Do I need deep learning?* The useful question is about **doors**: *which job would this skill let me apply for that I can't apply for today?*

Asked backwards, the question leads to months of learning things no employer will ask you about yet, while the three skills that would get you shortlisted stay half-learned. Asked the right way round, it turns an endless list of topics into a short, ordered plan with a job at the end of each step.

This chapter gives you that plan. It arranges the ten roles from Chapter 7 into a tree of tiers, shows exactly which skills each role needs, lets you see what each job looks like on an ordinary Tuesday, and teaches you to read a job description the way a hiring manager wrote it. By the end, you'll be able to name your next door, and the chapters of this book that open it.

In plain English

Picture an office building where every room is a job. Each room has a locked door, and each lock needs a particular set of keys.

- Each **key** is a skill: SQL, a spreadsheet, a BI tool, Python, statistics, the ability to write a clear requirement.
- Each **door** is a role you could be hired into. When you hold the keys a door needs, you become a credible applicant: someone a hiring manager would take seriously.
- The building has **floors**. The ground floor's keys open the first rooms. Higher floors need the lower keys *and* some new ones. Nobody takes the lift straight to the top.
- On one floor, two corridors branch off in different directions, one toward science and one toward engineering. They're on the same floor: neither is "above" the other.
- You **keep your keys** as you climb. A data scientist still uses SQL every week.
- A **job description** is the notice on a door, listing the keys it needs. Some are required; some are "nice to have".

The building is the **career tree**. The floors are **tiers**. The rest of this chapter is its floor plan.

8.1 Skills are keys; roles are doors

The draft of this book introduced one idea that stays at the center of this chapter: **learn skills for the doors they open, not because they're popular.**

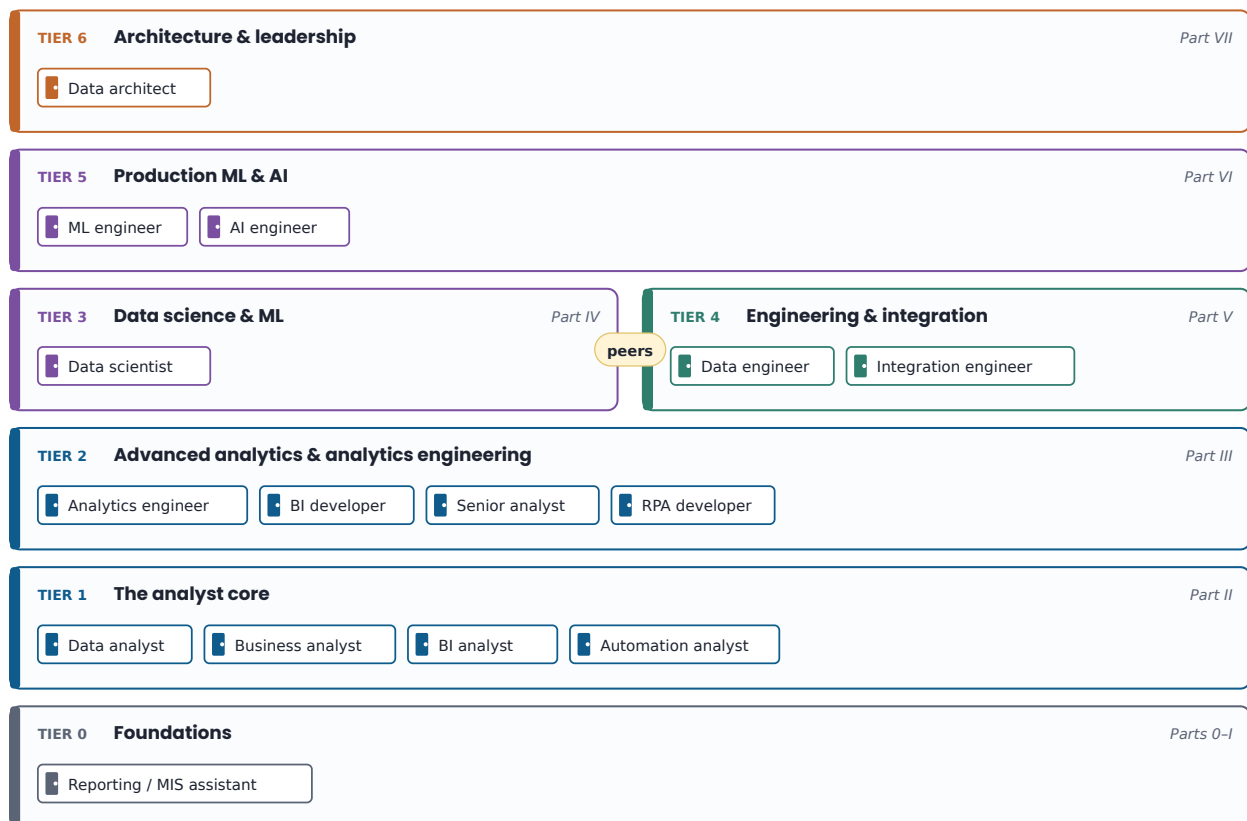
That idea changes how you study in three ways.

- **It gives every topic a reason.** You don't learn "data science" in the abstract. You learn the set of skills that makes you a credible applicant for *data scientist*, and each skill is a key you can see yourself adding.
- **It tells you when to stop.** When you hold the keys a door needs, you're ready to apply. You'll keep learning on the job, but you don't need to finish the whole book first.
- **It protects you from fashion.** A tool that's trending this year might open no door you want. A skill that appears in every job description for your target role is worth more than ten trending ones.

A **credible applicant** is someone who meets a role's must-have skills well enough to pass a screening and hold their own in a technical interview. It doesn't mean knowing everything, and it doesn't mean experience alone. Part VIII (from Chapter 68) shows what each interview round checks.

8.2 The tiers, and what each unlocks

The career tree has seven tiers, numbered from 0. They match the parts of this book, in order, so the book is the tree.



Read from the bottom up. Each role sits at the first tier that makes you a credible applicant; you keep every skill below it.

Figure 8.1 – The career tree as tiers and doors. Tiers 3 and 4 sit side by side: they're peers, and you can climb either one first.

Tier 0: Foundations (Parts 0 and I). What data is, how computers store it, how a business runs on it, numbers without fear, thinking like an analyst, and the map you're reading now. *Unlocks:* reporting assistant and MIS (management information system) roles, where you prepare regular reports mostly by hand. Few people stay here long, and the skills matter mainly because everything above rests on them.

Tier 1: The analyst core (Part II). Excel and Google Sheets in depth, SQL, cleaning data, visualization, Power BI, Python for analysis, spreadsheet and report automation, statistics, business metrics, working with stakeholders, and the professional toolkit. *Unlocks:* **data analyst**, **business analyst**, **BI analyst** (an analyst who mainly builds reports and dashboards), **automation analyst**, and the domain versions: sales, finance, marketing, operations, supply chain, and HR analyst. This is the single biggest unlock in the field. If you master only one tier, make it this one.

Tier 2: Advanced analytics and analytics engineering (Part III). Advanced SQL and data modeling, Python written as maintainable software, experiments and causal inference, dbt, the computer science you need, and the command line. *Unlocks:* **senior analyst**, **analytics engineer**, **BI developer** (who designs the data models and semantic layers behind dashboards, not only the visuals), and **RPA developer**. Tier 2 is the branch point: the skills here feed both branches above it.

Tier 3: Data science and machine learning (Part IV). The math under the models, the machine learning workflow, supervised and unsupervised learning, honest evaluation, forecasting, text, recommendations, and a first look at deep learning. *Unlocks:* **data scientist**, and specialist roles such as forecasting analyst or applied scientist.

Tier 4: Engineering and integration (Part V). Ingestion, pipelines and orchestration, data quality, distributed compute, warehouses and lakehouses, streaming, data activation and APIs, and the cloud. *Unlocks:* **data engineer**, **integration engineer**, and platform roles.

Tiers 3 and 4 are peers. They sit at the same height on the tree. The book teaches science first because many analysts move toward it first, but plenty of excellent careers go from tier 2 straight into engineering. This is the fix for the first edition's tier order, which put engineering before science while the book taught them the other way round. Now tier numbers, part numbers, and the tree agree, and the figure shows the two branches side by side.

Tier 5: Production ML and AI (Part VI). Deep learning in depth, generative AI and large language models, building AI applications, MLOps, LLMOps, intelligent automation, and industry cases. *Unlocks:* **ML engineer**, **AI engineer**, and MLOps and ML platform roles. Most people arrive here from tier 3 (with engineering added) or from tier 4 (with machine learning added).

Tier 6: Architecture and leadership (Part VII). Designing whole systems, trade-offs, architecture patterns, automation architecture and governance, security and privacy, the economics of data platforms, data strategy, and leading as an architect. *Unlocks:* **data architect**, ML or AI architect, and the management ladder up to head of data or chief data officer.

Where the automation roles sit

The automation and integration track from Chapter 7 doesn't fit neatly on one floor, because automation grows with every tier. Its roles appear at three levels:

Role	Tier	Why there
Automation analyst	1	Spreadsheet automation, scheduled reports, and low-code flows are analyst-core skills (Chapters 19–20)
RPA developer	2	Needs maintainable, tested automation and command-line habits (Part III), plus RPA tools
Integration engineer	4	Needs APIs, pipelines, data quality, and reverse ETL (Chapters 45–51)

How the tree works: three rules

These rules from the first edition still hold, and the rest of the chapter depends on them.

1. **You keep every skill below your tier.** Climbing doesn't mean leaving skills behind. They accumulate.
2. **A role sits at the first tier that makes you credible for it.** You don't need tier 6 to apply for a tier 1 job. Learn up to the door you want, apply, and keep climbing on the job.
3. **Pick your door, then learn backward.** Choose the role you want next, find its tier, and learn the keys that door needs. Ignore everything higher for now. It will still be there when you arrive.

Simplification note. Real hiring is messier than a tree. A small company might hire a "data scientist" who mostly does tier 1 work, and a large bank might require tier 2 skills for an analyst post. The tiers describe where a role's *must-have* skills are taught, which is what you need for planning. Section 8.5 shows how to read what a particular employer really wants.

8.3 The skills matrix: what each role really needs

The tiers tell you roughly where a role sits. The skills matrix tells you exactly which keys each door needs. It lists 17 skills against the ten roles, and marks each skill as **core** (needed to be hired), **useful** (helps, often listed as a plus), or **not needed** to get in.

Skill	DA	BA	BI	AE	DS	MLE	DE	AIE	AUT	ARC
Business and domain sense	●	●	○	○	●	○	○	○	●	●
Communication and storytelling	●	●	○	○	●	○	○	○	○	●
Spreadsheets (Excel, Sheets)	●	●	○	·	○	·	·	·	●	○
SQL	●	○	●	●	●	○	●	○	○	●
BI and dashboards	●	○	●	○	○	·	·	·	○	○
Statistics	○	·	·	·	●	○	·	○	·	·
Python	○	·	·	○	●	●	●	●	●	○
Spreadsheet and report automation	○	○	○	·	·	·	·	·	●	·
Requirements and process mapping	○	●	○	○	·	·	·	○	●	●
Data modeling (incl. dbt)	·	·	●	●	·	·	●	·	·	●
Machine learning	·	·	·	·	●	●	·	○	·	○
Pipelines and orchestration	·	·	·	○	·	●	●	○	○	●
Cloud and infrastructure	·	·	·	○	○	●	●	●	○	●
APIs and integration	·	·	·	·	·	○	●	●	●	●
LLMs and AI applications	·	·	·	·	○	○	·	●	○	○
Git and version control	·	·	○	●	○	●	●	●	○	○
System design and governance	·	·	·	○	·	○	○	○	○	●

Key: ● core · ○ useful · · not needed to get in. DA data analyst · BA business analyst · BI BI developer · AE analytics engineer · DS data scientist · MLE ML engineer · DE data engineer · AIE AI engineer · AUT automation analyst, RPA developer, or integration engineer · ARC data architect.

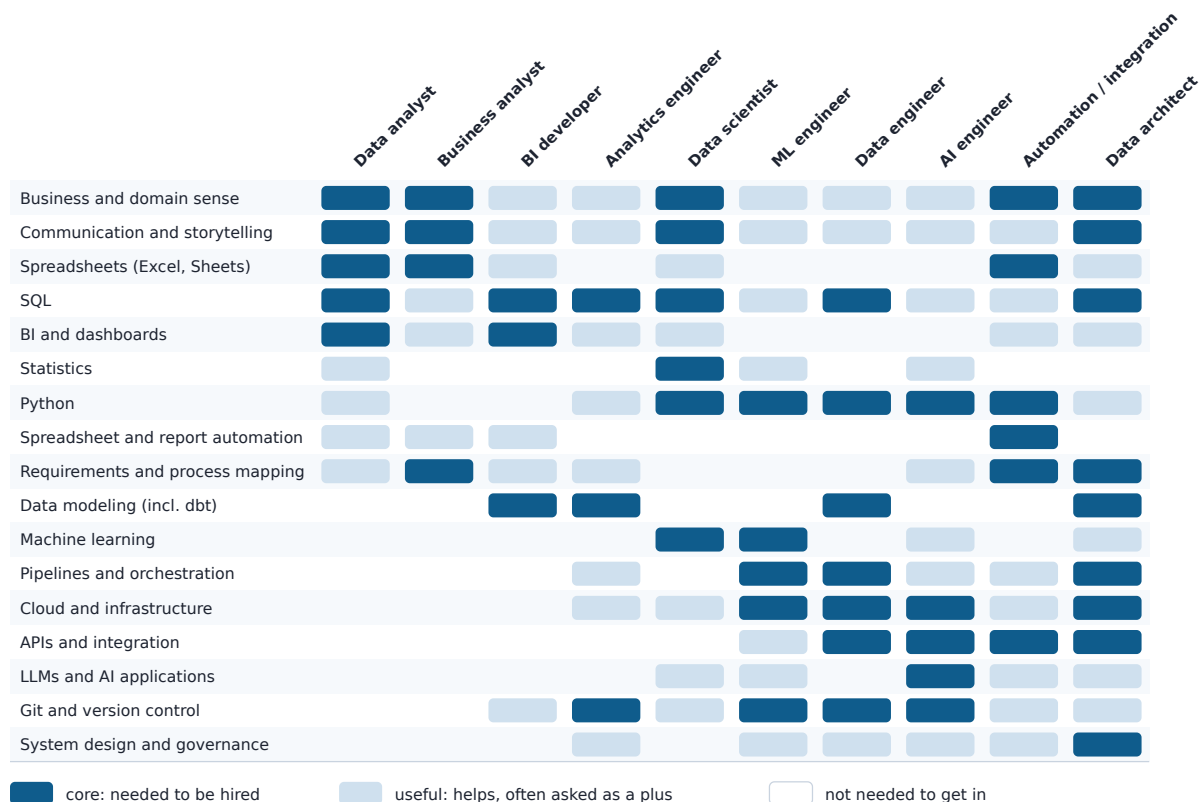


Figure 8.2 – The same matrix as a heatmap. Read down a column to see a role; read across a row to see which doors a skill helps open.

Reading the matrix

Read down a column to see what a role needs. The data analyst column has five core skills: business sense, communication, spreadsheets, SQL, and BI and dashboards. That's the tier 1 core. The data engineer column has seven: SQL, Python, data modeling, pipelines, cloud, APIs and integration, and Git. The data architect column is the darkest of all, with nine core skills, because the architect's job is to understand how every part fits.

Read across a row to see which doors a skill helps open. Three rows are worth noticing.

- **SQL** is marked core or useful for all ten roles. This is the promise from Chapter 7 in matrix form.
- **Machine learning** is core for only two roles, data scientist and ML engineer. If neither is your target, it can wait.
- **Python** is core for five roles (data scientist, ML engineer, data engineer, AI engineer, and the automation and integration roles) and useful for three more. It's the most important skill to add after the analyst core.

Compare two columns to see how far apart two roles are. Data analyst and BI developer share seven skills that each role needs at least a little of; data analyst and data engineer share only four (business sense, communication, SQL, and Python). That's why analyst-to-BI-developer is a short move and analyst-to-data-engineer takes longer.

Watch out: "useful" is not "optional forever." A useful skill often decides between two otherwise equal candidates, and it's usually the next key you'll need on the job. Treat the useful cells in your target column as your second list, after the core ones.

Try it. Pick the role you're most curious about. Count its core skills, and tick the ones you could use at work today. The unticked ones are your next keys.

8.4 A day in the life of each role

A matrix tells you what a job needs. It doesn't tell you what the job *feels* like. Here's an ordinary working day in each role. The people and companies are fictional; the shape of each day is typical, though every company differs.

Data analyst

Pooja works in the sales team of a consumer goods distributor. At 9:30 she checks that the morning's sales report refreshed and that yesterday's total matches the billing system. At 10:00 the regional manager asks why the West region missed its target; she spends two hours in SQL and a spreadsheet, finds that one large wholesaler delayed an order to next month, and writes a four-line answer. After lunch she builds a comparison of two discount schemes for Friday's review. At 4:30 a colleague reports a strange number on the dashboard, and she traces it to a customer entered twice. Most of her day is questions, SQL, checking, and explaining.

Business analyst

Aditya works for a hospital chain that is replacing its appointment system. His morning is a workshop with front-desk staff, mapping how appointments are booked, changed, and cancelled today, including the paper register nobody mentioned in the first meeting. He turns the notes into a process map and eight user stories, each with acceptance criteria. After lunch he pulls last quarter's cancellation data with a short SQL query to show that most cancellations happen by phone, which changes the design. The day ends with a call with the software vendor to agree what "reschedule" means. Most of his day is conversation, documents, and small, sharp pieces of analysis.

BI developer

Lakshmi works in a bank's reporting team. She starts by checking the overnight dataset refreshes in Power BI; one failed because a source table was renamed, so she fixes the query. Her main task is a new branch-performance dashboard: she designs the data model, writes measures, and makes sure the numbers match the finance team's official figures to the rupee. In the afternoon she reviews a colleague's report for slow visuals and confusing labels. Before leaving, she sets row-level security so branch managers see only their own branch. Her day is modeling, measures, performance, and access.

Analytics engineer

Rohan works at an online retailer. His morning starts with a failing data test: an "orders" model suddenly has duplicate order IDs. He finds that a source system began sending corrections as new rows, fixes the model, and adds a test so it can't happen silently again. Then he meets the marketing and finance analysts to agree a single definition of "new customer". He writes the change in dbt, documents it, and opens a pull request that a colleague reviews. In the afternoon he helps an analyst turn a 300-line query into three reusable models. His day is SQL, tests, Git, and agreeing definitions.

Data scientist

Meenal works for an insurance company. Her project is predicting which policyholders are likely to not renew. In the morning she checks whether a promising feature is really *leakage*: information the model wouldn't have at prediction time. It is, so she removes it, and the model's accuracy drops to a more believable level. After lunch she presents results to the retention team, explaining in plain language what the model can and can't do, and proposes a test on one region before any wider rollout. She ends the day writing up the experiment design. Her day is Python, statistics, careful doubt, and explanation.

ML engineer

Farhan works at a logistics company that uses a model to estimate delivery times. His morning alert says the model's errors have grown over the past week; he finds that a new warehouse opened and the model has never seen its routes. He schedules retraining with the new data and adds a monitoring check per warehouse. In the afternoon he packages a colleague's new model as a service, writes deployment tests, and sets up a way to roll back if it misbehaves. His day is code, monitoring, deployment, and reliability.

Data engineer

Sneha works for a fintech company. At 8:00 she checks the overnight pipelines; one loaded zero rows from the payments system because an API credential expired, so she renews it and reruns the load before analysts arrive. She spends the morning building a new pipeline that brings support tickets into the warehouse, with checks for missing fields and duplicates. After lunch she meets the payments team to agree a *data contract*: which fields they'll send, in what format, and how they'll warn her before changing it. Her day is pipelines, failures, quality checks, and agreements with system owners.

AI engineer

Karan works for a company that sells industrial parts. He's building an assistant that answers sales reps' questions from product manuals and price lists. In the morning he runs the assistant against a set of 150 test questions with known answers, finds that it invents a warranty period for one product line, and changes how documents are retrieved so the right manual is used. After lunch he adds a rule that any price in an answer must come from the price list, with a link. He ends the day reviewing how often reps click "not helpful". His day is Python, retrieval, evaluation, and guardrails.

Automation analyst or integration engineer

Nisha works in the operations team of a manufacturer. Her morning goes on a report that three planners rebuild by hand every Monday: she replaces it with a Power Query refresh and a scheduled flow that emails each planner their section in the body of an email. After lunch she investigates why yesterday's purchase orders didn't reach the supplier portal; an API call failed and nobody was alerted, so she adds retries and a failure email to the team inbox. She finishes by updating the automation inventory: what runs, when, who owns it. Her day is removing manual steps and making sure automations fail loudly, not silently.

Data architect

Vivek works for a retail group with six brands. His morning is a design review: a team wants to add a new tool for customer data, and he asks how it will match customer IDs with the existing warehouse, who will own the matching rules, and what it will cost in two years. After lunch he writes an *architecture decision record*: a one-page note of a decision, the options considered, and why one was chosen. He spends an hour with the security team on who may access sales data by region, and ends the day sketching next year's platform roadmap for the head of data. His day is questions, trade-offs, documents, and influence without direct authority.

Real-life example: the title you'll actually see. Many of these people's job titles don't match the role names above. Pooja's title might be "MIS executive", Nisha's "process excellence analyst", Lakshmi's "senior reporting analyst". Search job portals by the work ("Power BI", "dbt", "Power Automate") as well as by title.

8.5 Decoding a job description

A **job description** (JD) is written by a hiring manager, often edited by HR, and read by hundreds of applicants and, first, by software that filters CVs. Reading one well means separating what the manager needs from what got copied in from last year's posting.

Here is the full posting that Anita Rao and Meera Iyer wrote in Chapter 7's story. It's fictional, but it's typical of a real posting for a tier 1 role.

Data Analyst (Sales Analytics and Automation) Riverstone Supplies · head office, hybrid · 1–3 years' experience (freshers with strong projects considered)

About the role. You'll be Riverstone's first dedicated data analyst, working with the sales team and reporting to the Sales Head. You'll turn our sales and customer data into lists, dashboards, and reports that the team uses every week, and automate the reports we currently build by hand.

What you'll do

- Build and run a weekly list of at-risk customers for the sales reps.
- Build and maintain the sales dashboard in Power BI.
- Agree definitions of key numbers, such as "active customer" and net revenue, with the finance team.
- Automate recurring reports, including the Daily Sales Flash.
- Answer ad-hoc questions from sales leadership, clearly and on time.

Must have

- SQL: joins and aggregation; window functions a plus.
- Excel or Google Sheets: lookups and pivot tables.
- A BI tool, Power BI preferred.
- Experience automating a recurring report, with any tool.
- Clear written communication in English.

Nice to have

- Python (pandas).
- VBA or Google Apps Script.
- Forecasting or customer analytics.
- Experience with a CRM or ERP.

Reading it line by line

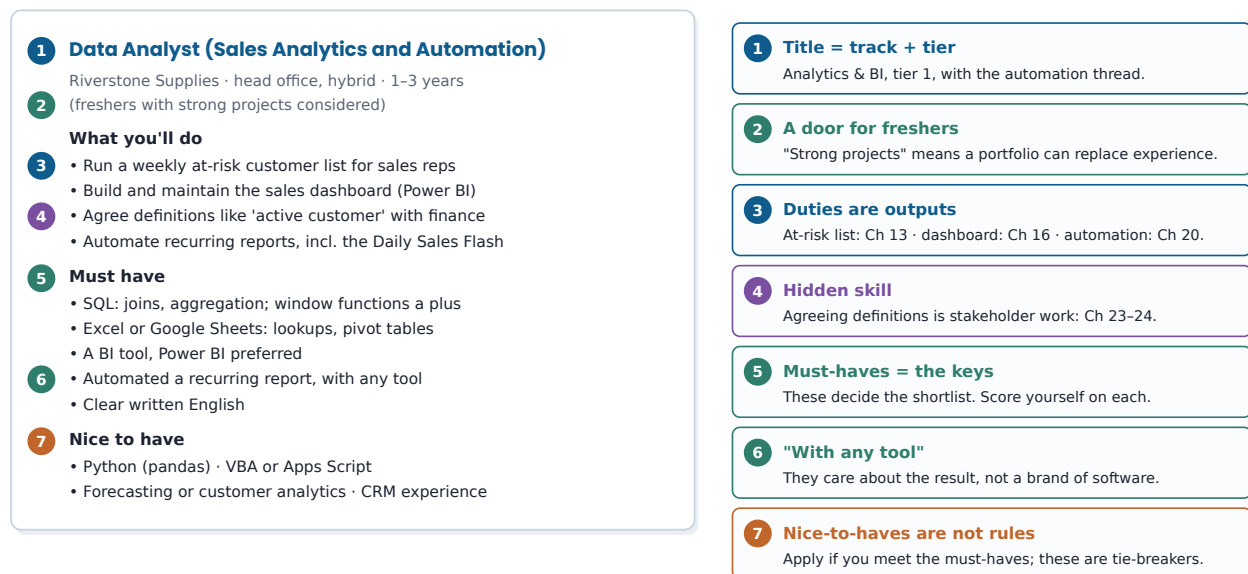


Figure 8.3 – Decoding a job description. Every line tells you something about the job, and a few lines tell you what the manager is worried about.

Work through a JD in this order.

1. **Title: find the track and the tier.** "Data Analyst" puts it on the analytics and BI track at tier 1. "Sales Analytics and Automation" tells you the domain (sales) and a thread (automation). The words in brackets often say more than the title.
2. **Experience: read the exception.** "1–3 years" is a guide; "freshers with strong projects considered" is a door. The manager is saying a portfolio can stand in for experience. Chapter 9 and Chapter 27 show how to build one.
3. **Duties: translate each into an output.** An at-risk customer list is Chapter 13's Pattern 6. A Power BI dashboard is Chapter 16. Automating the Daily Sales Flash is Chapters 19–20. When every duty maps to something you've built, you can talk about it in the interview.
4. **Find the hidden skill.** "Agree definitions with the finance team" isn't a tool. It's stakeholder work: negotiating a shared meaning across departments (Chapters 23–24). Hiring managers put lines like this in because they've felt the pain.
5. **Must-haves: these are the keys.** They decide the shortlist. Be strict with yourself here.
6. **Watch the qualifiers.** "Window functions a plus" means basic SQL is required and advanced SQL is a bonus. "Power BI preferred" means another BI tool is acceptable. "With any tool" means they care about the result: a recorded macro, an Apps Script trigger, or a Python script all count.
7. **Nice-to-haves: tie-breakers, not rules.** If you meet the must-haves, apply. Nobody expects every nice-to-have.

Scoring your fit

A quick, honest way to judge whether to apply is to score yourself. For each skill in the JD, give yourself **0** (can't do it yet), **1** (have done it with help, or in a course), or **2** (have done it on real or realistic data without help). Must-haves count double, because they decide the shortlist.

Take Farah Khan, a sales executive at Riverstone who is interested in this role (you'll meet her properly in "In the real world").

Skill	Type	Farah's score	Weight	Points
SQL	Must	1	2	2
Excel or Google Sheets	Must	2	2	4
A BI tool	Must	0	2	0
Automated a recurring report	Must	0	2	0
Written communication	Must	2	2	4
Python	Nice	0	1	0
VBA or Apps Script	Nice	1	1	1
Forecasting or customer analytics	Nice	0	1	0
CRM or ERP experience	Nice	2	1	2
Total				13

The maximum is 28: five must-haves at 2 points $\times$ 2 weight = 20, plus four nice-to-haves at 2 points $\times$ 1 weight = 8. Farah scores 13 of 28, or 46.4%. More useful than the total is her **must-have coverage**: her raw scores on the must-haves add up to $1 + 2 + 0 + 0 + 2 = 5$ out of a possible 10, which is 50%.

Reading it. Farah isn't ready to apply today, but she's much closer than she feels. Her gaps are specific: SQL to working level (Chapters 12–13), a BI tool (Chapter 16), and one automated report she can show (Chapters 19–20). Her strengths, spreadsheets, communication, and CRM experience, are exactly the ones that are hard to teach.

Simplification note. This scoring is a planning tool, not a formula employers use. Its value is that it forces you to be specific about each gap, and to see which gaps matter most.

Red flags in job descriptions

Some postings tell you more about the company than the job. Watch for these patterns.

- **A tool list longer than the duties.** Fifteen tools, from Excel to Kafka to TensorFlow, for a fresher role usually means nobody decided what the job is.
- **A senior title with junior pay or junior duties.** "Data scientist" whose duties are data entry and weekly MIS reports.
- **Every tier at once.** "Build dashboards, pipelines, ML models, and the cloud platform" for one person is four jobs.
- **No outputs at all.** If you can't tell what you'd produce in your first three months, ask in the interview.
- **Unpaid "test projects" that look like real work.** A reasonable take-home task is short and clearly an exercise; Chapter 82 shows what good ones look like.

None of these is always a reason to walk away. A small company with a messy JD might be a great place to learn. But they're questions to ask before you accept.

Interview extra point. Bring your decoded JD to the interview. When asked "Why this role?", name two duties from the posting, say what you've built that matches each, and ask one question about the hidden skill ("How are definitions like 'active customer' agreed today?"). Chapter 68 explains why this lands well with hiring managers.

8.6 What the roles pay, and how to read salary figures

Pay matters, and you deserve real numbers. You also deserve an honest warning: **salary figures online are rough, vary widely by source, and go stale within a year.** Treat everything in this section as a way to read salary data, with examples, not as a promise of what you'll earn.

Two kinds of salary source

Salary websites build their figures in two main ways.

- **Self-reported salaries.** People enter their own pay, title, and experience. PayScale works this way. The figures reflect real pay, but only from people who chose to report it.
- **Job postings.** The site collects the salaries advertised in job ads. Indeed's salary pages work this way. The figures reflect what employers advertise, which may differ from what people accept.

Because they measure different things, the two can disagree for the same role, and neither is wrong.

Three roles, as published in 2026

Here are figures for three roles, as shown on PayScale's India pages (updated in July 2026) when this chapter was written. **Base salary** is fixed yearly pay before bonuses; **total pay** adds bonuses and similar extras.

Role (PayScale title)	Salary profiles	Average total pay, under 1 year	Average total pay, 1-4 years	Base salary, 10th-90th percentile	Median base salary
Data Analyst	2,389	₹413,462	₹565,999	about ₹289,000 to ₹1 million	about ₹577,000
Data Scientist	1,267	₹595,255	₹1,005,147	about ₹312,000 to ₹2 million	about ₹1 million
Data Engineer	1,284	₹518,398	₹810,624	about ₹410,000 to ₹2 million	about ₹975,000

Source: PayScale India job pages for each title, pages updated 9–14 July 2026, retrieved 16 September 2026. PayScale rounds its percentile figures (for example "₹1m"), so those columns are approximate. Figures change often; check the live pages.

In Indian terms, a data analyst's average total pay in the first year works out to about ₹4.1 lakh, and a data scientist's average in years one to four to about ₹10.1 lakh.

A **percentile** tells you where a value sits in a sorted list. The 10th percentile is the value below which 10% of reported salaries fall; the 90th percentile is the value below which 90% fall. So "₹289,000 to ₹1 million" means the middle 80% of data analyst profiles reported base pay in that range. The **median** is the middle value: half earn less, half earn more.

Reading the table carefully

The pattern is clearer than the numbers. Roles that need more tiers of skill (science and engineering) show higher medians and faster early growth than the analyst role in this data. The data scientist median base salary (about ₹1 million) is roughly 1.73 times the data analyst median (about ₹577,000).

Don't read growth as a promise. Average total pay for data analysts with 1–4 years is 36.9% higher than for those under a year: $(₹565,999 \div ₹413,462 - 1) \times 100 = 36.9\%$. For data scientists the difference is 68.9%, and for data engineers 56.4%. But these are *different groups of people*, not the same people a few years later. People who left the field, or never reported, aren't in the data.

Sources disagree. For data analysts, Indeed's India page (based on 681 salaries from job postings, updated 30 August 2026) showed an average of ₹633,625 a year. PayScale's average base salary for the same title was ₹577,472. The difference is ₹56,153, with Indeed about 9.7% higher. That's not an error: one counts advertised pay across all experience levels, the other counts reported base salaries.

Some roles don't have reliable figures under their own title yet. Analytics engineer, AI engineer, and integration engineer are newer titles, and many people doing that work are listed under older ones (data engineer, software engineer, BI developer). Business analyst figures mix IT business analysts, finance business analysts, and more. For these, look at live job postings for the work you'd do, not only the title, and compare several sources.

CTC and in-hand pay

Indian offers are usually stated as **CTC** (cost to company): everything the employer spends on you in a year. CTC often includes the employer's provident fund contribution, variable or performance pay that isn't guaranteed, and sometimes insurance, gratuity, or one-time bonuses. Your **in-hand** (take-home) pay is what reaches your bank account each month after income tax, your own provident fund contribution, and other deductions. In-hand pay is always lower than $CTC \div 12$, and how much lower depends on how the offer is structured and on tax rules at the time. Always ask for the breakup, and ask which parts are fixed.

Watch out: what moves pay. City, company type, industry, and your specific skills can move pay more than the job title does. Two "data analyst" offers in the same month can differ by more than the whole range between roles in the table above. Compare offers on fixed pay, the work you'll do, and what you'll learn, not on the headline CTC alone. This is general information, not financial advice.

How to check salaries yourself

1. Look up the role on at least two sources that use different methods (for example PayScale and Indeed), and note the date each page was updated.
2. Filter by city and experience where the site allows it, and write down the number of salaries behind each figure. A figure built from 20 salaries means much less than one built from 2,000.
3. Read five live job postings for the role in your city and note any advertised ranges.
4. Ask people in the role, respectfully, what range is typical. Most people will share a range, not their own salary.
5. Treat the result as a range, and update it every six months.

8.7 Reader pathways: your route through this book

Not everyone climbs to the top, and you shouldn't read every chapter with the same care. This table suggests a route for each goal. "Read fully" means work through the chapters, exercises, and projects. "Skim" means read to understand the ideas and vocabulary. "Interview chapters" lists the Part VIII chapters to prepare with.

Goal	Read fully	Skim	Interview chapters
Complete beginner, exploring	Parts 0, I	Part II (first half)	—
Data analyst	Parts 0, I, II (all)	Part III (Ch 28, 30)	68, 69, 70, 71, 72, 73, 75, 78, 81, 82
Business analyst	Parts 0, I; Ch 10–16, 19–27	Ch 17–18	68, 69, 70, 71, 75, 76, 78, 81
BI developer	Parts 0, I, II; Ch 28, 32	Ch 45–49, 51, 63	68, 69, 70, 71, 77, 78, 81
Analytics engineer	Parts 0–III	Ch 45–49, 51	68, 69, 71, 72, 77, 81
Automation / integration engineer	Parts 0, I; Ch 10–20, 25, 29, 34, 45–47, 51, 58, 63	Ch 52, 55	68, 69, 70, 71, 72, 76, 77, 78, 81, 82
Data scientist	Parts 0–IV	Part V; Ch 53–56, 58	68, 69, 71–75, 79, 81, 82
Data engineer	Parts 0–III, V	Part IV (Ch 35–39); Ch 56, 63	68, 69, 71, 72, 77, 78, 81, 82
ML / AI engineer	Parts 0–VI	Part VII	68, 69, 71, 72, 74, 77, 78, 79, 81, 82
Data / ML architect	Everything	—	68, 69, 77, 78, 79, 80, 81
Already an analyst	Skim Parts 0–II; start fully at Part III	—	Per target role

Two things are worth noticing. First, **every route reads Parts 0 and I fully**, including this chapter, and every route except the beginner's goes through the analyst core. Second, the **automation and integration route** is the most spread out: it takes the spreadsheet and report automation chapters

from Part II, the software habits from Part III, the integration chapters from Part V, and the automation chapters from Parts VI and VII.

The automation thread, tier by tier

Whichever route you take, automation grows with you. At each tier, the same business need, getting the right data to the right people and systems without manual work, is met with better tools.

Tier	What automation looks like at this level	Taught in
Foundations	Seeing the manual steps hidden in a business process	Ch 3, 5
Analyst core	Refreshable spreadsheets; recorded macros, VBA, Office Scripts, and Google Apps Script; scheduled scripts; reports delivered by email, including in the email body; dashboard subscriptions and alerts; low-code flows	Ch 11, 16, 18, 19, 20
Business analyst	Mapping processes, finding and prioritizing automation opportunities, writing automation requirements	Ch 25
Advanced analytics and analytics engineering	Tested, version-controlled, maintainable automations; scheduled dbt jobs; command-line scheduling	Ch 29, 32, 34
Engineering and integration	Orchestrated pipelines, quality gates before delivery, pushing data back into business systems (reverse ETL, APIs, webhooks), integration platforms, RPA where there's no API	Ch 45-47, 51
Production ML and AI	Models and AI agents inside business workflows, with human approval where it matters	Ch 56, 57, 58
Architecture	Designing the whole source-to-action flow as one system, choosing the right tool for each job, and governing every automation	Ch 60, 63
Interviews	Spreadsheet automation questions; automation, integration, and report-delivery questions and design cases	Ch 70, 78

8.8 Entry routes: fresher, career switcher, internal move

Most people enter the tree at tier 1. How they get there depends on where they start.

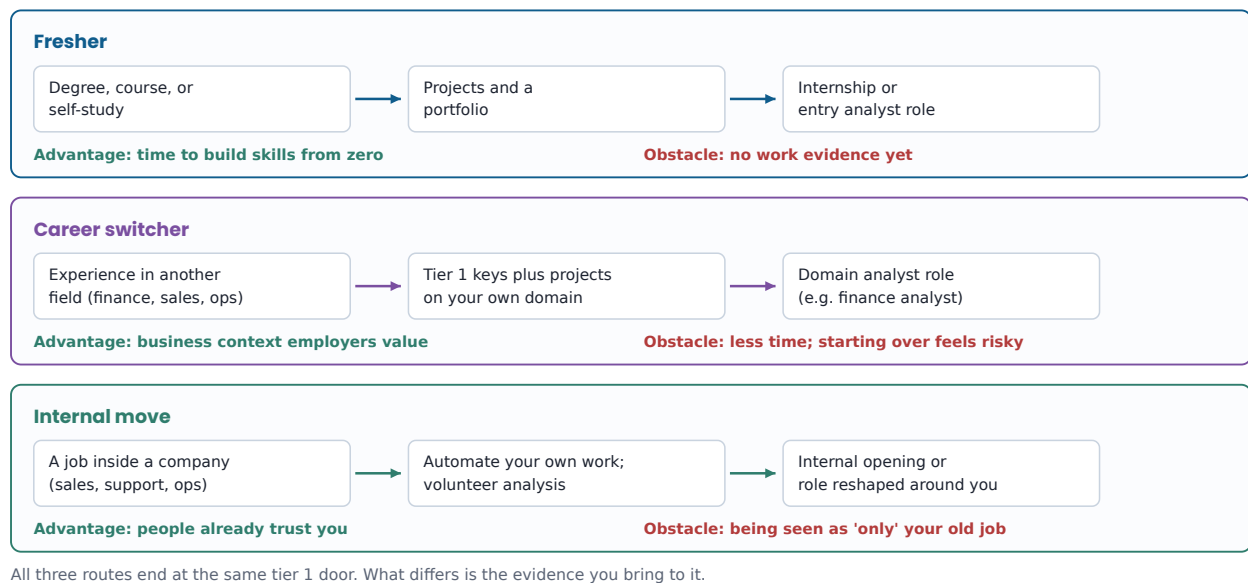


Figure 8.4 – Three entry routes. They end at the same door; what differs is the evidence you bring.

The fresher

A **fresher** is someone applying for their first job, usually straight from a degree or course. Your challenge is evidence: the employer can't see work you've done, so you have to show it.

- **Build the tier 1 keys properly**, not a little of everything. SQL, a spreadsheet, a BI tool, and one automated report beat a certificate in ten tools.
- **Make projects that look like work**. Use realistic data (the Riverstone datasets, public government data, or a business you know), answer a real question, and write up what you'd tell a manager. The projects at the end of every chapter in Part II are designed for this, and Chapter 27 turns them into a portfolio.
- **Take internships seriously**, including short or unpaid-but-fair ones at small companies, where you often get real data and real responsibility.
- **Apply to the doors with exceptions**. Postings that say "freshers with strong projects considered" are written for you.

The career switcher

A **career switcher** moves into data from another field: accounts, sales, operations, teaching, engineering, healthcare. Your challenge is time, and the feeling of starting over. You aren't starting over.

- **Your domain is your advantage**. An accountant who learns SQL and Power BI is a strong candidate for *finance analyst*, because they already understand what the numbers mean. Aim first for a data role in your own domain.
- **Build projects on your own domain's problems** (with made-up or public data, never your employer's confidential data). A former logistics coordinator's delivery-delay analysis tells a hiring manager more than a generic movie-ratings project.

- **Rewrite your experience as data work.** "Prepared monthly MIS reports for 12 branches" is analysis experience. Say what you produced and what decision it supported.
- **Plan in months, not weeks.** Chapter 9 is honest about how long this takes alongside a job.

The internal move

An **internal move** is changing into a data role inside the company where you already work. For many people this is the fastest route, and it's how many data teams in mid-sized companies begin.

- **Automate your own work first.** The report you rebuild every week is your first project, and the hours you save are visible to your manager.
- **Volunteer for analysis nobody owns,** such as a question your manager keeps asking, a list that's always out of date, or a number two teams disagree on.
- **Tell your manager what you're aiming for.** Managers can't support a move they don't know about, and they often know about openings first.
- **Expect the "old job" problem.** People may keep seeing you as "the person from sales". Evidence helps: a dashboard people use, an automation that saved hours, a problem you solved with data.

Real-life example: Meera's route. Meera Iyer (Chapter 1) joined Riverstone as a sales coordinator, not as an analyst. By Chapter 7 she was the person everyone asked for numbers, because she asked good questions about the data and fixed the reports she inherited. That's an internal move in progress: the job title hasn't changed yet, but the work already has.

8.9 Using the tree: pick your door, then learn backward

Put the chapter together and planning your next step takes four moves.

1. **Pick one door.** Choose the role you want next, not the one you want in ten years. Use Chapter 7 and section 8.4 to check that you'd enjoy the day, not only the title.
2. **Find its keys.** Take the role's column in the skills matrix (section 8.3), then check it against five real job descriptions for that role in your city (section 8.5).
3. **Score yourself and list the gaps.** Use the scoring in section 8.5. Order the gaps: must-haves first, then the useful skills that appear most often in the postings.
4. **Map gaps to chapters and projects.** Use the pathways table (section 8.7) to find where each gap is taught, and plan one project that shows each must-have skill.

Then apply when your must-have coverage is strong, and keep climbing on the job. When you're settled, pick the next door.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Learning tools because they're trending	Many half-finished courses; no door you're ready for	Pick one door and learn its keys first (section 8.9)
Trying to learn the whole tree before applying	Two years of study, no applications	A role sits at the first tier that makes you credible; apply when you get there
Treating tiers 3 and 4 as a ladder	Feeling you must do data science before engineering (or the reverse)	They're peers; choose by the work you enjoy
Skipping the analyst core	Models or pipelines built on questions you can't frame or data you can't check	Build tier 1 properly; every branch rests on it
Reading "nice to have" as "must have"	Never applying, because you don't meet every line	Score must-haves strictly; treat nice-to-haves as tie-breakers
Reading "must have" as "nice to have"	Many applications, few shortlists	Close must-have gaps before applying widely
Searching only by job title	Missing roles called "MIS executive" or "process excellence analyst"	Search by the work and tools too (section 8.4)
Comparing offers by headline CTC	Accepting the "bigger" offer and getting less in hand	Ask for the breakup; compare fixed pay and the work
Treating an online salary figure as a promise	Disappointment, or turning down fair offers	Use ranges from two sources, note the date and sample size (section 8.6)
Building generic projects	Portfolio looks like everyone else's	Use realistic business questions, ideally in your own domain
Switchers hiding their old career	CV looks like a fresher's	Rewrite past work as data work; your domain is an advantage
Waiting for an internal role to be announced	Openings filled before you hear of them	Automate your work, volunteer analysis, and tell your manager your goal

In the real world: Farah's internal move

Farah Khan had been a sales executive at Riverstone for four years. She knew the hospitality customers better than anyone: which caterers ordered before wedding season, which hotel buyer needed a call before Diwali. When Anita pinned the new *Data Analyst (Sales Analytics and Automation)* posting on the notice board, Farah read it three times and then walked past it.

Meera noticed. At lunch she asked, "Are you going to apply?"

"I'm a sales executive," Farah said. "I use Excel. That's not the same thing."

"Let's check," Meera said, and pulled the posting up on her laptop. They went through it the way section 8.5 describes. Farah scored herself: 2 for spreadsheets ("I run my whole customer list in Sheets, with lookups and a pivot"), 2 for written communication, 2 for CRM experience. Then 1 for

SQL, because she'd done an online course but never used it on real data; 1 for Apps Script, because she'd once recorded a macro to format her weekly report; and 0 for a BI tool, automating a report, Python, and forecasting.

Farah looked at the total, 13 of 28, and shrugged. "Less than half."

"Look at the must-haves instead," Meera said. "You're at half of those already. And look at *which* half you have. Anita can teach someone Power BI in a few months. She can't teach someone which caterers go quiet in the monsoon, and why."

They turned the gaps into a plan. Meera was careful not to promise a timeline; Farah had a full-time job, and the honest answer was months, not weeks.

- **SQL to working level:** Chapters 12 and 13 of this book, practiced on Riverstone's own practice databases, and then on questions from her own customers.
- **A BI tool:** Chapter 16, building a dashboard of her own region's customers.
- **One automated report she could show:** her weekly customer follow-up list, rebuilt as a Google Sheet that refreshes itself and emails her the list every Monday (Chapter 19).

Farah also asked the question she'd been avoiding. "What happens to my sales targets while I'm doing this?"

"Talk to Anita," Meera said. "Tell her you're aiming for the role. She'd rather grow someone who knows the customers than hire a stranger."

The conversation with Anita was shorter than Farah expected. Anita didn't promise her the job. The posting would stay open, and she'd interview other candidates on the same must-haves. But she agreed that Farah could spend Friday afternoons helping Meera with the at-risk customer list, and she asked Farah to bring the Monday email to the next sales review once it worked.

Six weeks later, the email worked. It wasn't sophisticated: a query, a sheet, and a time-driven trigger. But every Monday at 9 a.m. it listed Farah's customers who hadn't ordered in their usual rhythm, and in the first month it caught two she would have missed. At the sales review, Vikram asked if his team could have the same email.

What made this work.

- **Farah scored herself against the real posting,** not against a vague idea of "being technical". Her gaps became three specific items.
 - **She used her domain as an advantage,** and built her first project on her own customers.
 - **She made her goal visible,** and turned an internal move from a hope into an agreement about what evidence would count.
 - **Nobody over-promised.** The role wasn't guaranteed, and the timeline was honest. The evidence she built would count here or anywhere else.
-

Tools

- **A spreadsheet** (Excel or Google Sheets) for your skills matrix and gap score, or paper if you prefer.
 - **Two or more job portals.** Save postings as PDFs or screenshots; they disappear.
 - **Two salary sources that use different methods**, such as PayScale (self-reported) and Indeed (job postings), plus your city filter. Note the date on every figure you record.
 - **Companion files:** none for this chapter. The skills matrix values are listed in `checks/ch08_check.py` and in `figures/make_figs08.py`, if you'd like to rebuild or change it.
-

The project: your door plan

Goal: a two-page plan for your next role, grounded in real job postings. It's the most useful document you'll write in Part I, and a good one makes interviews easier, because you'll already know what the employer wants and what you can show.

Option A: your own target. Use the role you want next, in the city or remote market you'll apply to.

Privacy reminder. If you're planning an internal move, don't include confidential details about your employer, colleagues, or internal postings in anything you share publicly.

Option B: Farah's plan. Use the Riverstone posting in section 8.5 and Farah's scores, and write the plan she'd bring to Anita.

Steps

1. **Pick one door.** Write the role and one sentence on why, using the day in the life in section 8.4.
2. **Collect five job descriptions** for that role. For Option B, use the Riverstone posting plus four real postings for similar roles.
3. **Decode each one** with the seven steps in section 8.5. Highlight must-haves in one color and nice-to-haves in another.
4. **Build a skills table** in a spreadsheet: skills down the side, postings across the top. Mark each skill as must, nice, or absent in each posting. Add a column counting how many postings list each skill as a must-have.
5. **Score yourself** 0, 1, or 2 on every skill, and calculate your score and must-have coverage for the posting you like most.
6. **List your gaps in order:** must-haves that appear in most postings first.
7. **Map each gap to chapters and one project** that would prove the skill, using the pathways table in section 8.7.
8. **Check pay** for the role using the steps at the end of section 8.6, and record the range, sources, dates, and sample sizes.
9. **Choose your entry route** (section 8.8) and write the three things you'll do differently because of it.

Stretch goals

- Repeat steps 4–6 for a second, higher door, and mark which gaps the two doors share.
 - Ask one person in your target role to review your plan, and note what they'd change.
 - Set a date, three months out, to re-score yourself against the same postings.
-

You've got it when...

- ☐ You can explain "skills are keys, roles are doors" and why it changes what you learn next.
 - ☐ You can name the seven tiers in order, say which part of the book teaches each, and explain why tiers 3 and 4 are peers.
 - ☐ You can place each of the ten roles at the first tier that makes someone credible for it.
 - ☐ You can read your target role's column in the skills matrix and list its core skills.
 - ☐ You can describe an ordinary day in at least three roles, and say which you'd enjoy.
 - ☐ You can decode a job description into title, duties as outputs, hidden skills, must-haves, and nice-to-haves.
 - ☐ You can score your fit and calculate your must-have coverage.
 - ☐ You can read a salary table with percentiles, and explain why two sources disagree.
 - ☐ You can explain CTC versus in-hand pay.
 - ☐ You can choose a reading pathway through this book for your goal.
 - ☐ You've written down your next door and its three most important gaps.
-

Recap

- **Skills are keys; roles are doors.** Learn skills for the roles they unlock, not because they're popular.
- The **career tree** has seven **tiers** that match the book's parts: foundations (0), the analyst core (1), advanced analytics and analytics engineering (2), data science (3), engineering and integration (4), production ML and AI (5), and architecture and leadership (6). **Tiers 3 and 4 are peers.**
- A role sits at the **first tier that makes you a credible applicant**, and you keep every skill below it.
- The **skills matrix** shows each role's **core** and **useful** skills. SQL helps open every door; machine learning is core for only two.
- A **day in the life** matters as much as the matrix: choose work you'd enjoy on an ordinary day.
- **Decoding a job description** means finding the track and tier, turning duties into outputs, spotting hidden skills, and separating **must-haves** from **nice-to-haves**. **Must-have coverage** is the most useful fit score.
- **Salary figures** differ by method (self-reported versus job postings), date, sample size, city, and company. Read **percentiles** and **medians**, compare two sources, and know **CTC** from **in-hand** pay.
- **Reader pathways** show which parts of this book to read fully or skim for each goal, and automation grows with every tier.

- There are three **entry routes**: **fresher**, **career switcher**, and **internal move**. They end at the same door with different evidence.
-

Practice exercises

Warm-up

1. List the seven tiers in order, with the part of the book that teaches each. Which two tiers are peers?
2. Place each role at the first tier that makes someone credible for it: (a) automation analyst, (b) analytics engineer, (c) ML engineer, (d) data architect, (e) business analyst, (f) integration engineer.
3. Using the skills matrix in section 8.3: (a) For which roles is machine learning a core skill? (b) For which roles is Python a core skill? (c) Which skill is core or useful for every role?

Core

4. Classify each line from a job posting as a must-have, a nice-to-have, a hidden skill, or a signal about who can apply: (a) "Experience with Tableau or Power BI." (b) "Exposure to Python is an advantage." (c) "You'll work with regional managers to agree monthly targets." (d) "Final-year students with internship experience may apply." (e) "Strong SQL is essential."
5. Rahul scores himself against the Riverstone posting in section 8.5. Must-haves: SQL 0, spreadsheets 2, BI tool 1, automated a report 1, written communication 1. Nice-to-haves: Python 1, VBA or Apps Script 0, forecasting 2, CRM or ERP 2. Calculate his total score out of 28, the percentage, and his must-have coverage. Which single gap would you tell him to close first, and why?
6. Using the PayScale table in section 8.6, calculate how much higher average total pay is for data engineers with 1–4 years than for those under a year, as a percentage. Then give one reason why this isn't the pay rise a new data engineer should expect after a few years.
7. A friend says: "*Indeed says data analysts earn ₹633,625 on average, but PayScale says ₹577,472. One of them must be wrong.*" Explain why both can be right, giving two differences between the sources.
8. Explain the difference between CTC and in-hand pay in three sentences, as you would to a friend comparing two offers.
9. Using the reader pathways table, list what someone aiming to be a BI developer should read fully, what they should skim, and which interview chapters they should prepare with.

Stretch

10. Here's a posting: "*Data Scientist (Fresher). Must know Excel, SQL, Power BI, Tableau, Python, R, Spark, Hadoop, Kafka, AWS, Azure, GCP, TensorFlow, PyTorch, and LLMs. Duties: daily data entry of invoices and preparing weekly MIS reports for the accounts team.*" List at least three red flags, say what the job really is (role and tier), and write two questions you'd ask before applying.

- l1. Neha, a sales executive with strong Excel skills and three years of customer contact, wants to become a data analyst. Recommend an entry route and write a five-step plan using section 8.8 and section 8.9. Include which chapters of this book close which gaps.
- l2. Using the skills matrix, count the skills that data analyst and BI developer both need at least some of (core or useful in both columns), and do the same for data analyst and data engineer. What do the two counts suggest about which move is shorter, and which skills make the difference?

Think about it (no calculation needed)

- l3. The tree says a role sits at the first tier that makes you credible. Why is that more useful for planning than placing each role at the highest tier its best practitioners reach?
- l4. Could someone skip tier 1 and go straight into data engineering, for example from a software engineering job? What would they gain, and what would they risk missing?

Key terms

key · door · credible applicant · career tree · tier · MIS (management information system) · BI analyst · senior analyst · skills matrix · core skill · useful skill · leakage · data contract · architecture decision record · job description (JD) · must-have · nice-to-have · hidden skill · must-have coverage · self-reported salary · base salary · total pay · percentile · median · CTC (cost to company) · in-hand pay · reader pathway · fresher · career switcher · internal move

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 9, How Expertise Actually Forms**, is honest about how long each tier takes, and shows how to practice deliberately, build a portfolio as you learn, find feedback and mentors, and get through plateaus.
 - **Part II, The Analyst (Chapters 10–27)**, is tier 1: the keys for data analyst, business analyst, BI analyst, and automation analyst. **Chapter 27** turns your projects into a portfolio.
 - **Chapters 23–24** teach the stakeholder skills behind "hidden skill" lines in job descriptions; **Chapter 25** covers the business analyst track in depth.
 - **Chapter 66** returns to careers from the other side of the desk: building and hiring for data teams.
 - **Part VIII, Chapter 68, How Data Hiring Works**, explains the interview rounds for each role, CVs, portfolios, and referrals. **Chapter 81** covers behavioral questions and offer conversations, including negotiating on CTC. **Chapter 82** shows what fair take-home assignments look like.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. Tier 0, foundations (Parts 0 and I); tier 1, the analyst core (Part II); tier 2, advanced analytics and analytics engineering (Part III); tier 3, data science and machine learning (Part IV); tier 4, engineering and integration (Part V); tier 5, production ML and AI (Part VI); tier 6, architecture and leadership (Part VII). **Tiers 3 and 4 are peers.** The common wrong answer puts data engineering at tier 3, as the first edition did; the tier numbers now follow the book's parts.
2. (a) Automation analyst: tier 1. (b) Analytics engineer: tier 2. (c) ML engineer: tier 5. (d) Data architect: tier 6. (e) Business analyst: tier 1. (f) Integration engineer: tier 4. The trap is (a) and (f): both are on the automation track, but at different tiers, because an integration engineer needs the pipeline and API skills of Part V.
3. (a) Data scientist and ML engineer. (b) Data scientist, ML engineer, data engineer, AI engineer, and the automation and integration roles. (c) SQL, which is core for six roles and useful for the other four.
4. (a) Must-have, with a qualifier: either BI tool is acceptable. (b) Nice-to-have: "an advantage". (c) Hidden skill: negotiating targets with managers is stakeholder work, not a tool. (d) Signal about who can apply: a door for students with internship evidence. (e) Must-have: "essential".
5. Must-have points: $(0 + 2 + 1 + 1 + 1) \times 2 = 5 \times 2 = 10$. Nice-to-have points: $1 + 0 + 2 + 2 = 5$. Total **15 of 28 = 53.6%**. Must-have coverage: 5 of 10 = **50%**, the same as Farah's, even though his total is higher, because his extra points come from nice-to-haves. Close **SQL** first: it's a must-have, he scores 0, and it's the one skill every role on the tree uses. His forecasting score won't get him shortlisted without it. The common wrong answer is to celebrate the higher total; must-haves decide the shortlist.
6. $(\text{₹}810,624 \div \text{₹}518,398 - 1) \times 100 = \textbf{56.4\%}$ higher. It isn't a promised rise because the two figures come from **different groups of people**, not the same people over time. The 1–4 year group also only includes people still in the role who chose to report, and pay depends heavily on city, company, and skills. Any one of these is a good reason.
7. Both can be right because they measure different things. (1) **Method:** Indeed's figure comes from salaries advertised in job postings; PayScale's comes from salaries people report themselves. (2) **What's counted:** the PayScale figure quoted is average base salary, while advertised figures may include other pay. The sources also differ in **sample** (681 postings versus 2,389 profiles), **date** (updated 30 August 2026 versus July 2026), and the mix of experience and cities. The gap is ₹56,153, about 9.7%, which is small next to the range within either source.
8. A sample answer: "CTC is everything the company says it spends on you in a year, including things like the employer's provident fund share and bonuses that aren't guaranteed. In-hand pay is what actually reaches your bank account each month, after tax and your own deductions, so it's always less than CTC divided by 12. When you compare offers, ask for the breakup and compare the fixed, monthly in-hand amount, not only the CTC headline."
9. **Read fully:** Parts 0, I, and II, plus Chapters 28 and 32. **Skim:** Chapters 45–49, 51, and 63. **Interview chapters:** 68, 69, 70, 71, 77, 78, and 81.

10. Red flags: (1) a **tool list far longer than the duties**, covering every tier from spreadsheets to cloud to LLMs; (2) a **senior title with junior duties**: invoice data entry and MIS reports aren't data science; (3) a **fresher role expected to know tools from tiers 1–5**, which nobody entering the field has. The real job is closest to a **reporting or MIS assistant (tier 0 to tier 1)**, with some data entry. Useful questions: "What would I produce in my first three months?" and "Which of the listed tools are used by the team today?" Other good questions ask about who the role reports to, or whether there's a path into analysis. The job might still be a reasonable first step if the pay and learning are fair, but not under the belief that it's a data science role.

11. Recommended route: **internal move** if Neha's company has analysis work she can take on, otherwise **career switcher** into a sales analyst role, using her customer knowledge as her advantage. A five-step plan: (1) **Pick the door:** data analyst, ideally sales analytics. (2) **Find the keys:** the data analyst column (business sense, communication, spreadsheets, SQL, BI) checked against five postings. (3) **Score and list gaps:** likely SQL (Chapters 12–13), a BI tool (Chapter 16), and one automated report (Chapters 19–20); her spreadsheets and business sense are strengths. (4) **Build evidence on her own domain:** automate her weekly customer report and build a dashboard of her region, using non-confidential or practice data if she'll show it publicly. (5) **Make it visible:** tell her manager her goal, volunteer for an analysis nobody owns, and re-score herself in three months. Any plan that orders must-have gaps first and ties each to a project is acceptable.

12. Data analyst and BI developer: 7 shared skills (business and domain sense, communication and storytelling, spreadsheets, SQL, BI and dashboards, spreadsheet and report automation, requirements and process mapping). **Data analyst and data engineer: 4** (business sense, communication, SQL, and Python). Analyst to BI developer is the shorter move: the analyst already has most of what's needed, and the main new core skill is data modeling. Analyst to data engineer needs several skills the analyst column doesn't have at all: data modeling, pipelines and orchestration, cloud, APIs and integration, and Git.

13. Because it tells you when you can apply. If each role were placed where its best people end up, a data analyst would sit at tier 6 (some senior analysts have architect-level judgment), and every beginner would conclude they need years of study before applying for anything. Placing roles at the first credible tier gives each door a clear, reachable set of keys, and the rest can be learned on the job, which is how most people climb.

14. Yes, it's possible, and common. A software engineer already has much of what data engineering needs: Python, Git, cloud, APIs, and software habits. They'd gain speed. They'd risk missing the tier 1 skills that make data engineering *useful*: business sense, understanding what the numbers mean, knowing what analysts need, and the habit of checking a total against a source. A data engineer without them can build fast, reliable pipelines that deliver the wrong numbers on time. The fix is to learn the analyst core alongside the move, not to skip it.

Chapter 9. How Expertise Actually Forms

Chapter at a glance

You will learn to: estimate how long a stage of your learning will take, from hours and your real weekly schedule · explain why a long timeline is an advantage, not a punishment · combine the three ingredients of expertise: study, projects, and feedback over time · design deliberate practice sessions instead of hours of passive study · build a portfolio from the work you do while learning · find feedback, peers, and mentors, and ask for help in a way people say yes to · recognize a plateau in your own practice log and change what you do about it.

Before you start: Chapter 8 (the career tree, and choosing your next door).

Time needed: 2–3 hours, including the exercises. The project runs alongside your learning for 12 weeks.

Tools: a spreadsheet or notebook for a practice log. Nothing to install.

Practice data: Farah Khan's fictional 12-week SQL practice log, and the time estimates printed at the start of Chapters 12 and 13. Every calculation shown is checked.

Why this matters

Chapter 8 told you *what* to learn and in what order. This chapter is about *how long* it takes, and *how* the learning turns into ability. It's the chapter that protects you from two opposite mistakes.

The first mistake is quitting too early. In month four, you'll have worked hard, finished chapters, and still feel slow when you open a real dataset. If you believe the promise that anyone can become a data scientist in twelve weeks, that feeling will look like failure. It isn't. It's the normal middle of learning something difficult.

The second mistake is stopping too soon. In year two, you'll be good at your job, and the work will feel comfortable. Comfort is pleasant, and it's also the point where many people stop improving without noticing.

Both mistakes come from not knowing how expertise forms. Once you know, you can plan realistically, practice in a way that works, collect evidence as you go, and recognize a plateau as a signal to change your practice rather than a reason to give up.

In plain English

Think about learning to cook.

- You can read recipes and watch videos. That's **study**. It teaches you what should happen, but on its own it doesn't get dinner on the table.
- You start cooking real meals, with real ingredients that don't behave like the video. The dal sticks to the pan; the rice is undercooked. That's a **project**, and it's where study turns into skill.
- Your family tells you the dal needs less salt. An aunt watches you and says you're adding the tadka too early. Over months, you cook the same dishes many times and start adjusting without thinking. That's **feedback and time**, and it's what turns skill into judgment.
- Some evenings you cook what you already know. That's comfortable, and you don't get better. Other evenings you try one dish that's slightly beyond you, and pay attention to exactly what went wrong. That's the difference between **naive practice** and **deliberate practice**.
- After a few months, your cooking stops improving for a while, however often you cook. That's a **plateau**. The fix is usually to change what you practice, not to cook more of the same.
- If you photograph your best dishes and write down what you changed, you end up with a record of what you can do. That's a **portfolio**.

Everything in this chapter is a version of learning to cook, applied to data work.

9.1 The honest timeline

Let's be plain about time, because a great deal of advertising is not.

Reaching real expertise in this field takes years, not weeks. Becoming a job-ready analyst (tier 1) is a matter of months of steady work for most people who start from zero. Each tier above adds more study, and the top of the tree isn't mainly about study at all. Architects earn their judgment by designing real systems, watching some of them fail, and being responsible when something broke on a Friday night.

No book, course, or bootcamp changes that arithmetic. A course can make your study efficient and your projects well chosen. It can't give you the years of feedback that turn skill into judgment. Anyone who promises to compress a decade into ten weeks is describing a brochure, not a career.

Estimating your own timeline

"Months" and "years" are vague, and vague plans get abandoned. So estimate with numbers you can check.

Every chapter in this book starts with a *Time needed* estimate. Chapter 12, *Databases & SQL Foundations*, says 19–23 hours of reading and practice. Chapter 13, *SQL for Real Analysis*, says 15–20 hours. Together, that's **34–43 hours** to work through the book's two core SQL chapters properly, with exercises.

Now divide by the hours you can *really* give each week, not the hours you wish you had:

Hours per week	Weeks for Chapters 12 and 13
6	$34 \div 6 = 5.7$ to $43 \div 6 = 7.2$ weeks

Hours per week	Weeks for Chapters 12 and 13
10	$34 \div 10 = 3.4$ to $43 \div 10 = 4.3$ weeks

At 6 hours a week, a realistic amount alongside a full-time job, the SQL chapters alone take about six to seven weeks. Part II has eighteen chapters. Working through all of it at that pace is a matter of many months, and that's before the extra practice that makes SQL fluent rather than familiar.

Watch out: chapter hours are not fluency hours. Finishing Chapters 12 and 13 means you can write the queries they teach. Being quick and confident on a messy real dataset takes more: many more questions, answered on data you didn't design. Plan for the chapter hours, then plan for practice beyond them. Section 9.4 shows how to make those hours count.

Try it. Write down the hours you gave to learning last week, counted truthfully (not the hours you had planned). Multiply by four. That's your realistic monthly budget, and the number to plan with.

9.2 Why the long timeline is good news

Here's the reframe that makes the long timeline an advantage instead of a punishment. It's kept from the first edition of this book, because it's still true.

Because it's slow, it's defensible. A skill anyone can pick up in a weekend, everyone picks up in a weekend, and it earns no premium. The very thing that makes this career demanding is what makes it valuable. The years are the moat around your work.

Because it compounds, early effort pays for decades. SQL you learn in month one is still in use in year ten. Chapter 8's skills matrix showed SQL helping to open every door on the tree. Foundations don't expire; they become what everything else stands on.

Because it's layered, you're employable the whole way up. You don't wait years for the payoff. Finish tier 1 and you can be hired as an analyst. Finish tier 4 and you can be an engineer. Every tier is a real job, not only a checkpoint, so you earn while you climb.

9.3 The three ingredients of expertise

Expertise in data work isn't made from study alone. It's an alloy of three ingredients, and leaving any one out gives a weak result.

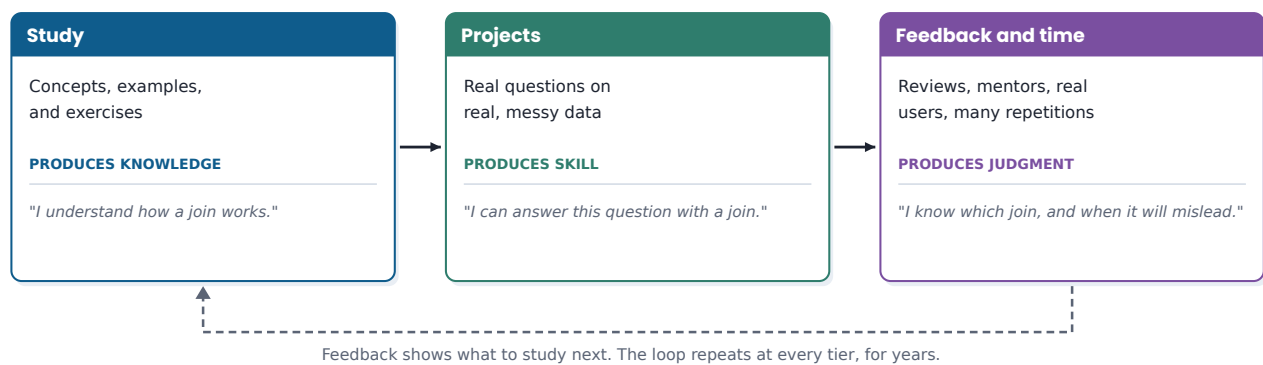


Figure 9.1 – The three ingredients, and what each produces. Most people who stall have plenty of the first and too little of the other two.

1. **Study** gives you **knowledge**: the concepts, from books like this one. It's necessary, but on its own it produces people who can *talk* about data and not *do* it.
2. **Projects** turn knowledge into **skill**: building real things with real, messy data, where nothing matches the example. This is the ingredient most people shortchange, and the one that matters most.
3. **Feedback and time** turn skill into **judgment**: code reviews, mentors, stakeholders who question your numbers, and many repetitions. It can't be rushed.

The three form a loop. Feedback shows you what you don't understand, which tells you what to study next, which you then use on the next project.

People who plateau are usually over-supplied with the first ingredient. They read endlessly and build rarely, because reading feels productive and building feels exposing. Don't be one of them. **Study enough to start building, then build.**

9.4 Deliberate practice

Not all practice is equal. Two people can each spend 100 hours "doing SQL" and end up far apart, because of *how* they spent those hours.

What deliberate practice is

The idea of **deliberate practice** comes from research on expert performers, most associated with the psychologist K. Anders Ericsson, whose studies of musicians in the early 1990s compared how the best and the merely good had practiced. Deliberate practice is practice designed specifically to improve performance, rather than practice that repeats what you can already do.

It has five features.

- **A specific goal.** Not "do some SQL", but "write a query that keeps customers who have never ordered".

- **Difficulty slightly beyond your current ability.** Hard enough that you might fail, not so hard that you can't tell why.
- **Full attention,** for a limited time. Short, focused sessions beat long, distracted ones.
- **Immediate feedback.** You find out quickly whether you were right, and *why* you weren't.
- **Repetition with refinement.** You try again, changed by what the feedback told you, and you come back to it later.

Naive practice is the opposite: doing what's comfortable, without a goal or feedback, and counting the hours.

	Naive practice	Deliberate practice
Goal	"Do some SQL tonight"	"Write a LEFT JOIN that keeps never-ordered customers"
Difficulty	What already feels comfortable	Slightly beyond what you can do unaided
Attention	Half-watching a video, phone nearby	Full focus for a short, fixed block
Feedback	None, or "it ran, so it's right"	Compare with a known answer; find why it differs
Repetition	Move on after one success	Redo it tomorrow without looking; vary it
Record	Nothing written down	A log: what was hard, what to try next

Same hours, different results: deliberate practice spends them where you're weakest, and checks every attempt.

Figure 9.2 – Naive and deliberate practice. The hours can be identical; what you do in them is not.

What the research does and doesn't say

You may have heard that expertise takes "10,000 hours". That number was popularized by Malcolm Gladwell's book *Outliers* (2008), drawing on Ericsson's research. Ericsson himself later objected that the number was an average from one group of musicians, not a threshold, and that the *kind* of practice mattered far more than the count.

Later research also found that practice isn't the whole story. A 2014 meta-analysis by Brooke Macnamara, David Hambrick, and Frederick Oswald, combining many earlier studies, found that the amount of deliberate practice explained 26% of the differences in performance in games, 21% in music, 18% in sports, 4% in education, and less than 1% in professions. The authors concluded that deliberate practice is important, but not as important as had been argued. Researchers still disagree about how to measure practice fairly, so treat the exact percentages with care.

Two honest conclusions follow for you.

- **Practice matters, and its quality matters most.** Nobody in the research became good without a great deal of practice.
- **Practice isn't everything, especially at work.** In professions, what you practice *on* (real problems, real consequences, real feedback) and who you learn *from* matter a great deal. That's why projects and feedback sit beside practice in the three ingredients, and why the rest of this chapter covers them.

Simplification note. The research above studied chess, music, sport, education, and various professions, not data work specifically. Its lessons transfer as sensible guidance, not as measured facts about analysts.

Deliberate practice for data skills

Here are practice methods that fit the five features, and how this book supports them.

Method	What you do	Where the book helps
Predict, then run	Before running a query or formula, write down the result you expect. Then run it and explain any difference.	<i>Predict-the-result</i> prompts in exercises; the "trap" examples in Chapters 12–13
Rebuild without looking	The day after a worked example, rebuild it from the question alone, without the book open.	Every "plain question first" example
Retrieval before rereading	Before rereading a section, write down what you remember. Then check.	<i>Recap</i> and <i>Key terms</i> sections
Spaced repetition	Return to a skill after a few days, then after a week or two, instead of all at once.	<i>Warm-up</i> exercises, and later chapters that reuse earlier skills
Vary the problem	Answer the same question on a different table, or a different question with the same technique.	<i>Core</i> and <i>Stretch</i> exercises; the pattern library in Chapter 13
Hunt for your mistakes	Keep a list of every mistake you make and its cause. Review it weekly.	<i>Common mistakes</i> tables in every chapter
Explain it aloud	Explain a result to someone else, or to a notebook, as if to a manager.	"What to tell Anita" sections

Research on learning has repeatedly found that testing yourself (retrieval) and spreading practice over time (spacing) produce more durable learning than rereading and cramming, even though rereading feels more productive. That feeling is the trap: the comfortable method feels like progress precisely because it asks so little of you.

Watch out: AI assistants can turn practice into watching. If an assistant writes the query every time you're stuck, you practice reading answers, not writing them. Use the assistant *after* your own attempt: to compare, to explain a difference, or to suggest a harder variation. Chapter 6 covers learning with AI assistants without letting them think for you.

9.5 Building a portfolio as you learn

A **portfolio** is a small collection of work that shows what you can do. For a fresher or career switcher, it often matters more than a certificate, because it answers the question a hiring manager really has: *can this person do the work?*

The most common mistake is treating the portfolio as something you build *after* learning. Build it *while* you learn. Every chapter project in this book is designed to become a portfolio piece, and Chapter 27 turns the best of them into a finished analyst portfolio.

What makes a good portfolio piece

A strong piece reads like a short piece of real work, not a tour of a tool. It fits on one page, with the details linked.

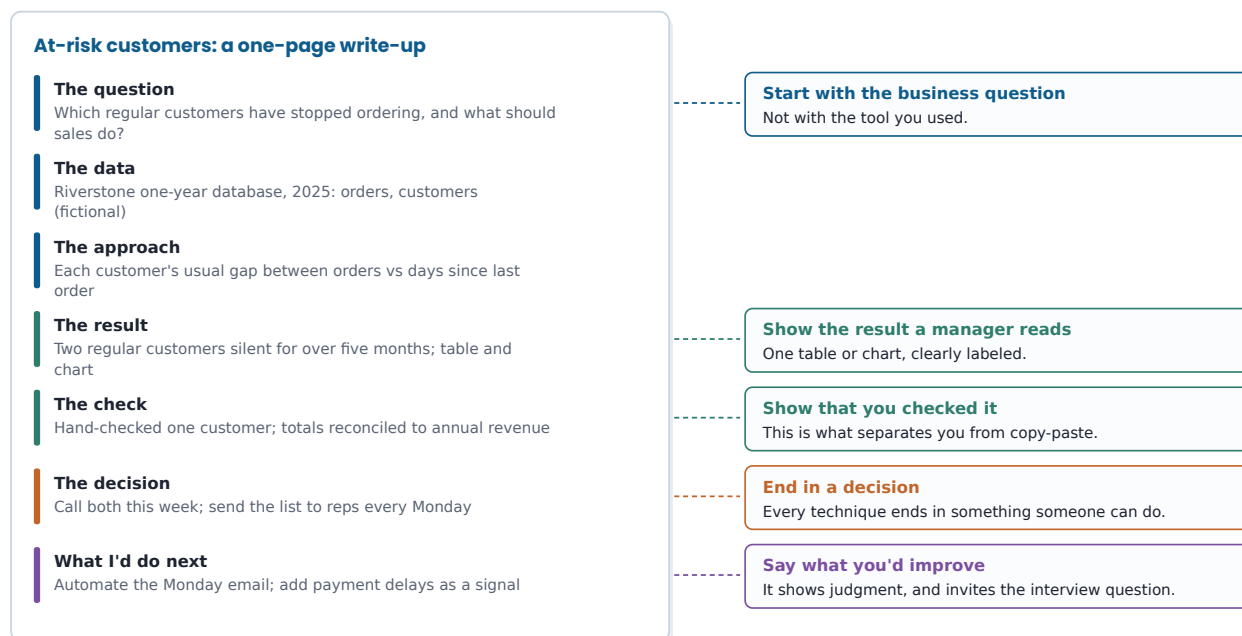


Figure 9.3 — The anatomy of a portfolio piece. The example uses Chapter 13's at-risk customer pattern on the Riverstone one-year database.

1. **The question.** One sentence a manager would ask, in business words.
2. **The data.** What you used, where it came from, and its limits.
3. **The approach.** How you answered it, in plain words, with the code linked.
4. **The result.** One clear table or chart.
5. **The check.** How you know it's right: a hand-checked row, a reconciled total.
6. **The decision.** What someone should do because of it.
7. **What you'd do next.** How you'd improve or automate it.

A portfolio that grows with you

Stage	Pieces worth building	From chapters
Foundations	A classified personal dataset; a process map of a real workflow	1, 3, 7
Analyst core	A cleaned dataset with a data-quality note; a SQL analysis answering three business questions; a dashboard; one automated report	12-20, 27
Advanced analytics	A tested dbt project; an A/B test analysis	30, 32

Stage	Pieces worth building	From chapters
Science or engineering	An end-to-end model with honest evaluation, or a monitored pipeline	44, 46–47
Architecture	A design document with decision records	60, 63

Practical rules

- **Quality over count.** Three pieces that each answer a real question beat ten tutorial copies.
- **Use realistic or public data,** never your employer's confidential data. The Riverstone datasets, public government data, and your own records (anonymized) are all fine.
- **Put it where people can open it in one click:** a GitHub repository with a clear README, a shared folder, or a simple web page. Chapter 26 covers Git and GitHub.
- **Keep the write-up short,** and make the first paragraph readable by a manager who never opens the code.
- **Update it as you climb.** Retire early pieces when better ones replace them.

9.6 Finding feedback and mentors

Feedback is the ingredient you can't give yourself entirely. You can check whether a query runs; it takes someone else to tell you that the question was wrong, or that your chart hides the point.

Where feedback comes from

Different sources give different kinds of feedback. Use several.

Source	What it's good for	Limits
Answer keys and known totals	Is this result correct? Every exercise in this book has a worked answer.	Only for problems with a known answer
The data itself	Reconciling to a trusted total, hand-checking a row	Doesn't tell you if the question was right
AI assistants	Explaining an error; reviewing your code after you've tried	Can be confidently wrong; not a substitute for checking
Study partners and peer groups	Swapping solutions, explaining to each other	Peers may share your blind spots
Online communities	Specific, well-asked technical questions	Slow; answers vary in quality
Colleagues and stakeholders at work	Whether your analysis helps real decisions	You have to show work early, before it's polished
Code review	Readability, correctness, habits	Needs a team or a willing reviewer
Mentors	Direction, judgment, career choices, what to learn next	Their time is limited; their experience is one path

Asking for help well

People say yes to questions that respect their time. A good request for help has five parts.

1. **What you're trying to do**, in one sentence.
2. **What you tried**, with the exact code or formula.
3. **What happened**, with the exact error or output.
4. **What you expected**, and why.
5. **A small example** someone can reproduce, with invented or public data.

Compare "My SQL join isn't working, please help" with "I'm trying to list every customer, including those with no orders. I used a LEFT JOIN with `WHERE o.status <> 'Cancelled'`, and customers with no orders disappear. I expected them to stay with blank order columns. Here are two small tables that show it." The second question is often answered in minutes, and writing it frequently reveals the answer before you send it. (Chapter 12, section 12.10, explains this exact trap.)

Finding a mentor

A **mentor** is someone further along the path who gives you occasional guidance. You don't need a famous one. Someone two or three years ahead of you in your target role is often more useful, because they remember the steps.

- **Start with a specific, small request**, not "will you be my mentor?" For example: "Could I have 20 minutes to ask how you moved from reporting into analytics engineering?"
- **Look close to home first**: senior colleagues, former classmates, alumni of your college, people who answered your questions in a community.
- **Do the work between conversations**. Arrive with what you tried since last time, and one specific question. Mentors keep helping people who use the help.
- **Give something back**. Share a useful article, thank them with a result ("your suggestion about the Monday email worked"), and help people behind you. Explaining to others is also excellent practice.
- **Have more than one**. Different people help with different things: one with SQL habits, another with career decisions.

Real-life example: feedback before it's finished. Many new analysts polish a dashboard for two weeks before showing anyone, and discover in the first meeting that it answers the wrong question. Experienced analysts show a rough version after a day: "Is this the question you meant?" Early feedback feels exposing, and it saves weeks.

9.7 Handling plateaus

Almost everyone who learns a difficult skill hits **plateaus**: stretches where effort continues and visible progress stops. The first edition of this book saved this topic for its closing chapter. It's here now, because you'll meet your first plateau long before the end of the book.

What a plateau looks like in data

Farah Khan, the Riverstone sales executive from Chapter 8, kept a simple log while working through the SQL chapters. Each week she recorded her practice minutes and took a **weekly check**: ten new problems at the same level of difficulty, solved without help, scored out of 10. A fixed check matters, because it measures progress on comparable problems, not on whatever she happened to practice.

Week	1	2	3	4	5	6	7	8	9	10	11	12
Practice minutes	180	200	210	220	240	260	270	240	240	250	240	250
Weekly check (of 10)	3	4	5	6	6	6	6	6	7	8	8	9

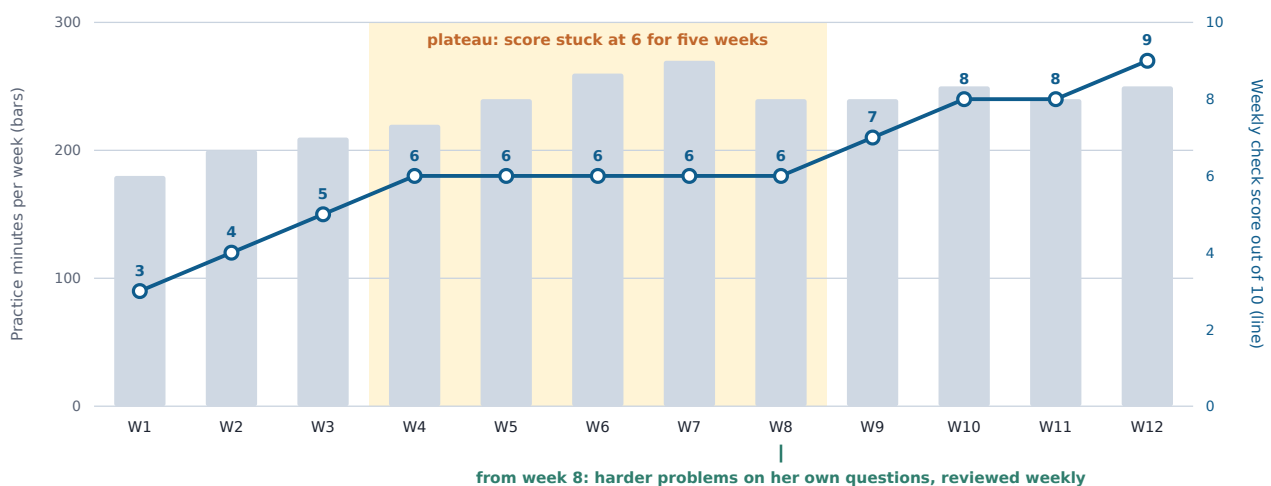


Figure 9.4 – Farah's practice log. During the plateau she practiced more, not less. The score moved again only when she changed how she practiced.

Reading it.

- In weeks 1–4, Farah practiced 810 minutes in total, and her score rose from 3 to 6.
- In weeks 4–8, she practiced 1,230 minutes, and her score didn't move. She responded the way most people do: she practiced *more*. Her weekly minutes rose from 220 in week 4 to 270 in week 7, an increase of 22.7%, with no gain.
- In week 8, she changed *what* she practiced (the story in "In the real world" explains how). In weeks 9–12, she practiced 980 minutes, fewer than in the plateau weeks, and her score rose from 6 to 9.

Over the 12 weeks, she practiced 2,800 minutes, about 46.7 hours: slightly more than the 34–43 hours the book estimates for Chapters 12 and 13. That's consistent with section 9.1: the chapter hours get you through the material, and fluency takes a little more.

Simplification note. Farah's log is fictional and deliberately tidy, to make the pattern clear. Real logs are noisier: a bad week, a holiday, a harder check. Look for a flat stretch over several weeks, not a single low score.

Why plateaus happen

- **The practice became comfortable.** Early on, everything is new, so any practice is deliberate. Later, you drift toward problems you can already solve, because solving them feels good.
- **The difficulty is wrong.** Problems that are too simple teach nothing; problems far too hard give no usable feedback.
- **You're missing one underlying idea.** A single misunderstanding (how NULLs behave, what a row represents) can block a whole family of problems.
- **Tool-hopping.** Switching to a new course or tool every few weeks resets you to the beginner stage, which feels like progress but isn't.
- **Life.** Work pressure, illness, family, and fatigue are real. Some flat weeks are rest, not failure.

What to do about a plateau

1. **Measure before you worry.** Without a fixed check, you can't tell a plateau from a feeling. Start logging.
2. **Change the practice, not only the amount.** More of the same rarely breaks a plateau. Raise the difficulty, switch to your own real questions, or add a new kind of feedback.
3. **Find the missing idea.** Review your mistakes list: do several errors share one cause? Go back to that section and rebuild its examples without looking.
4. **Get outside feedback.** Ask a peer or mentor to watch you solve one problem aloud. They'll often spot a habit you can't see.
5. **Teach it.** Explaining a topic to someone else exposes the parts you only half understand.
6. **Rest deliberately.** A planned week off, followed by a return to the same check, is better than grinding tired.
7. **Don't switch doors because of a plateau alone.** Choose a different role because you'd enjoy its work more (Chapter 8), not because the current skill has gone flat for a month. Every door has its plateaus.

9.8 What this means for how you use this book

Every remaining chapter gives you study *and* a project, on purpose, because the two belong together. Treat the projects as the real curriculum and the prose as the briefing before each one.

In practice, that means:

- **Do the exercises before reading the answers**, and predict results before running code.
- **Keep a practice log** from Part II onward: minutes, a weekly check, and your mistakes list.
- **Turn at least one project per part into a portfolio piece** with the seven-part write-up.
- **Show your work early** to at least one other person each month.
- **Expect plateaus**, and treat them as instructions to change your practice.

Be patient with the third ingredient, feedback and time. It's the one this book can't hand you. Only the work, and the years, can.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Believing a timeline advertised by a course	Feeling like a failure in month three or four	Estimate from chapter hours and your real weekly hours (section 9.1)
Planning with the hours you wish you had	Plans that collapse in the second week	Count last week's real hours; plan with those
Studying without building	Can explain a JOIN, can't answer a question with one	Start a small project as soon as you've learned enough to begin
Rereading instead of testing yourself	Material feels familiar but won't come back without the book	Write what you remember first, then check (retrieval practice)
Counting hours instead of designing practice	Many hours, little change	Use the five features of deliberate practice (section 9.4)
Letting an AI assistant solve every hard step	Can read solutions, can't write them	Attempt first; use the assistant to compare and explain
Building the portfolio "later"	Nothing to show when a good posting appears	Turn chapter projects into one-page pieces as you go
Portfolio of tutorial copies	Every piece looks like everyone else's	Start each piece with a real business question and end with a decision
Asking vague questions for help	Slow or no answers	Use the five-part request: goal, attempt, result, expectation, small example
Waiting until work is polished to show it	Weeks spent on the wrong question	Show a rough version early and ask "is this what you meant?"
Asking a stranger "will you be my mentor?"	Polite silence	Ask a specific, small question; do the work between conversations
Practicing more to break a plateau	More minutes, same results (Figure 9.4)	Change the difficulty, the problems, or the feedback
Switching courses or tools at every plateau	Always a beginner at something	Stay with the skill; change the practice

In the real world: Farah's week seven

By the seventh week of her SQL practice, Farah had stopped enjoying it.

She'd started well. The Riverstone practice databases made sense to her, because she knew the customers behind the rows. Her weekly check had climbed from 3 to 6 in four weeks. Then it stuck. Week 5: 6. Week 6: 6. In week 7 she practiced 270 minutes, the most yet, including two late nights after a long sales day, and scored 6 again.

On Friday afternoon, while they worked on the at-risk list, she told Meera she was thinking of switching to a Power BI course instead. "Maybe SQL isn't for me. I've plateaued."

"Can I see your log?" Meera asked.

Farah opened the sheet. Meera looked at the minutes column, then at the problems Farah had been practicing. Nearly all of them were from the warm-up and core exercises of Chapter 12, which Farah had now done two or three times each.

"You're not bad at SQL," Meera said. "You've become very good at these exercises. You know the answers. Your check has new problems, and that's where you're stuck."

She asked Farah to solve one of the week-7 check problems aloud. Farah got halfway and stopped: the problem needed customers with no orders, and her join kept dropping them. Meera recognized it immediately; she'd tripped over the same thing when she started. "Every one you missed last week has the same shape, doesn't it?"

Farah checked her mistakes list. Eight of the twelve problems she'd got wrong in weeks 5 to 7 involved customers or products that didn't appear in another table. It wasn't a plateau in SQL. It was one idea, the difference between an inner and a left join and where the filter goes, that she'd been working around instead of learning.

They changed three things.

- **Harder, real problems.** Instead of repeating chapter exercises, Farah wrote five questions a week about her own customers, the kind Anita actually asked, and answered them on the practice data.
- **One missing idea, fixed properly.** She went back to Chapter 12, section 12.10, rebuilt its LEFT JOIN examples without looking, and did its trap examples until she could predict every result.
- **Weekly feedback.** Every Friday, she showed Meera one query and explained it aloud, in the ten minutes before they started on the at-risk list.

She also cut her minutes back. The late nights stopped.

Week 8's check was 6 again, and Farah nearly gave up on the new plan. Week 9 was 7. Week 10 was 8. By week 12 she scored 9, having practiced fewer minutes in weeks 9 to 12 than during the plateau. And one of her "own questions" (*which hospitality customers ordered before last year's wedding season but not this year?*) turned into the first portfolio piece she was proud of, written up on one page with the check and the decision.

What made this work.

- **Farah had a log with a fixed check**, so the plateau was a pattern she could see, not a feeling.
 - **Meera looked at how Farah practiced**, not only how much.
 - **The fix was a change of practice**: harder, real problems; one underlying idea fixed; weekly feedback. Not more hours, and not a new course.
 - **The practice produced evidence.** The problems she practiced on became a portfolio piece.
-

Tools

- **A practice log**, in a spreadsheet or notebook. Columns: date, minutes, what you practiced, weekly check score, mistakes and their causes.
 - **A weekly check**: a fixed set of new problems at a steady difficulty. The exercises in later chapters, used for the first time, work well; don't reuse problems you've already practiced.
 - **A place for your portfolio**: a GitHub account (Chapter 26 shows how to use it), a shared folder, or a simple free web page.
 - **A timer**, for short focused practice blocks.
 - **Companion files**: none for this chapter. Farah's log is listed in `checks/ch09_check.py` and `figures/make_figs09.py` if you'd like to chart your own the same way.
-

The project: a 12-week learning system

Goal: a simple system that makes your learning measurable, turns it into evidence, and brings you feedback. Start it now and run it alongside Part II.

Option A: your own learning. Use the next skill on your door plan from Chapter 8.

Privacy reminder. If any practice uses data from your job, keep it private and anonymize anything you show others. Use practice or public data for portfolio pieces.

Option B: Farah's next 12 weeks. Plan Farah's next skill after SQL, Power BI (Chapter 16), using her door plan and this chapter.

Steps

1. **Set your weekly budget.** Count last week's real learning hours. Multiply by 12 for your 12-week budget, and compare it with the *Time needed* of the chapters you plan to cover. Adjust the plan, not the arithmetic.
2. **Build your log** with the columns in Tools.
3. **Design your weekly check:** ten new problems at a steady level, or a timed task (for example, "build this chart from raw data in 30 minutes").
4. **Plan your practice blocks** using at least three methods from the table in section 9.4.
5. **Choose one portfolio piece** to build during the 12 weeks, and outline its seven parts now.
6. **Name your feedback sources:** one peer or study partner, one community, and one person further along the path you'll ask a specific question.
7. **Run it for 12 weeks.** Every Sunday, fill in the log and take the check.
8. **At week 6 and week 12,** chart minutes and scores in the style of Figure 9.4, and write three sentences: what's working, whether you see a plateau, and what you'll change.

Stretch goals

- Keep a mistakes list and group mistakes by cause at week 6. Which single idea would fix the most?
 - Explain one topic to someone else each month, and note what you couldn't explain.
 - Publish your portfolio piece and ask two people for feedback on the write-up, not the code.
-

You've got it when...

- ☐ You can estimate how long a stage of your learning will take from chapter hours and your real weekly hours.
 - ☐ You can explain why a long timeline is defensible, compounding, and layered.
 - ☐ You can name the three ingredients of expertise and what each one produces.
 - ☐ You can describe the five features of deliberate practice and turn a vague goal into a specific one.
 - ☐ You can say what the research on practice does and doesn't show, including why "10,000 hours" is misleading.
 - ☐ You practice by predicting results and rebuilding examples without looking, not by rereading.
 - ☐ You can outline a portfolio piece in the seven parts, and you've started one.
 - ☐ You can write a request for help that's likely to be answered quickly.
 - ☐ You've identified at least two feedback sources and one possible mentor.
 - ☐ You keep a practice log with a fixed weekly check, and can recognize a plateau in it.
 - ☐ You know what to change when you hit a plateau, and why "practice more" usually isn't it.
-

Recap

- **Expertise takes years, not weeks.** Estimate your own timeline with numbers: chapter hours divided by the hours you can really give each week.
- The long timeline is an advantage: skills that take years are **defensible**, they **compound**, and every tier is a real job, so you're employable as you climb.
- Expertise combines three ingredients: **study** (knowledge), **projects** (skill), and **feedback and time** (judgment). Most people who stall have too much of the first.
- **Deliberate practice** has a specific goal, difficulty slightly beyond your ability, full attention, immediate feedback, and repetition with refinement. **Naive practice** counts hours.
- Research shows practice matters but isn't everything; the popular **10,000-hour** rule misreads it. In professions, real problems and real feedback matter a great deal.
- **Retrieval** and **spacing** beat rereading and cramming, even though rereading feels more productive.
- Build a **portfolio** while you learn: a few one-page pieces with a question, data, approach, result, check, decision, and next step.

- Get **feedback** from several sources, ask for help with a five-part request, and find **mentors** with small, specific asks.
 - A **plateau** is flat results despite steady effort. Measure it with a fixed **weekly check**, then change the practice rather than only adding hours.
-

Practice exercises

Warm-up

1. Name the three ingredients of expertise, and what each one produces.
2. Rewrite each vague practice goal as a specific, deliberate one: (a) "Get better at Excel." (b) "Practice SQL this weekend." (c) "Learn Power BI."
3. Classify each activity as naive or deliberate practice, and say why: (a) rereading Chapter 12 for the third time; (b) writing down the expected output of a query before running it; (c) watching a two-hour tutorial while replying to messages; (d) rebuilding yesterday's worked example from the question alone.

Core

4. Arjun can study 45 minutes a day, five days a week. How many hours will he have after 8 weeks? Is that enough to work through Chapters 12 and 13 at the book's estimate of 34–43 hours? If not, how many hours short of the lower estimate is he?
5. Compare two plans over a year: (a) 20 minutes every day of the year; (b) one 3-hour session every Sunday for 52 weeks. Calculate the hours in each. Which plan has more hours, and which would you recommend for learning SQL? Give a reason from section 9.4.
6. In Farah's log, her weekly check score went from 3 in week 1 to 9 in week 12. By what percentage did her score increase? Then explain why her *minutes* in weeks 5–7 are not evidence that she was working badly.
7. Rewrite this request for help using the five-part structure from section 9.6: "Pivot table not working in Google Sheets, totals are wrong, help!!" Invent reasonable details.
8. Outline a portfolio piece in the seven parts from section 9.5, for this question: "Which products sell best to hospitality customers, and should we change what we promote to them?"

Stretch

9. A learner's weekly check scores over ten weeks are 4, 5, 5, 6, 6, 6, 6, 5, 6, 6, while their practice minutes rose every week. Describe what the log shows, list two likely causes from section 9.7, and propose three specific changes.
10. Design one week of deliberate practice (five 40-minute sessions) for someone who has finished Chapter 13 and struggles with window functions. For each session, give the goal, the method, and the source of feedback.
11. A friend says: "Deliberate practice only explains a small share of performance in professions, so practice doesn't matter for data jobs." Using section 9.4, explain what's wrong with that conclusion,

and what the research does suggest.

Think about it (no calculation needed)

- l2. Why might a plateau be *more* likely after several months of learning than in the first few weeks?
 - l3. A career switcher has 8 hours a week and wants to spend all of them on courses until they "know enough" to start a project. What would you tell them, and why?
 - l4. What can a mentor give you that an AI assistant can't, and what can an AI assistant give you that most mentors can't?
-

Key terms

honest timeline · study · project · feedback · knowledge · skill · judgment · deliberate practice · naive practice · 10,000-hour rule · meta-analysis · retrieval practice · spaced repetition · portfolio · portfolio piece · mentor · plateau · practice log · weekly check · mistakes list

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Part II, The Analyst (Chapters 10–27)**, is where you'll use this chapter first. Start your practice log with Chapter 10.
 - **Chapter 6, Setting Up to Learn**, covers your study setup, a sample 6-month analyst plan, and learning with AI assistants without letting them think for you.
 - **Chapter 12, section 12.10**, teaches the LEFT JOIN trap that stalled Farah.
 - **Chapter 26** covers Git and GitHub, where your portfolio can live; **Chapter 27** turns your projects into a finished analyst portfolio.
 - **Chapter 83, The Long Game**, returns to learning over a whole career, including the plateaus of later years.
 - **Part VIII, Chapter 68, How Data Hiring Works**, shows how portfolios are read in hiring, and **Chapter 81** helps you turn your projects and plateaus into strong behavioral interview answers.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. Study produces knowledge (understanding the concepts). **Projects** produce skill (being able to do the work on real, messy data). **Feedback and time** produce judgment (knowing which approach to use, and when it will mislead).

2. Sample answers; any specific, checkable goal is acceptable. (a) "Build a pivot table of Riverstone revenue by month and category, and reconcile its grand total to the total in the source data." (b) "Write three queries that keep customers with no orders, predicting each result before running it." (c) "Build a one-page Power BI report with revenue by month and a slicer by segment, from the Chapter 16 dataset, in 60 minutes." The common weak answer names a topic ("learn LEFT JOINS") instead of a task with a checkable result.

3. (a) Naive: rereading feels productive but doesn't test recall or give feedback. (b) Deliberate: a specific prediction with immediate feedback when the result differs. (c) Naive: divided attention and no feedback. (d) Deliberate: retrieval (rebuilding from memory) with feedback (compare with the worked example).

4. $45 \times 5 \times 8 = 1,800$ minutes = **30 hours**. That's **not enough** for 34–43 hours: he's **4 hours short** of the lower estimate. He could extend the plan by about one more week, or add a longer weekend session. The point of the exercise is to adjust the plan to the arithmetic, not to hope.

5. (a) $20 \times 365 = 7,300$ minutes $\approx$ **121.7 hours**. (b) $3 \times 52 =$ **156 hours**. Plan (b) has more hours, but plan (a) is usually better for learning SQL, because practice spread over many days (spacing) and frequent short, focused sessions produce more durable learning than one long weekly block, where attention fades in the third hour. The best answer might combine them: short daily practice plus a longer weekly project session. Either recommendation is acceptable with a reason from section 9.4.

6. From 3 to 9 is an increase of 6 points: $(9 \div 3 - 1) \times 100 =$ **200%**. Her rising minutes in weeks 5–7 (240, 260, 270) aren't evidence of bad work: she was putting in *more* effort. The log shows that the *kind* of practice had stopped working (she was repeating familiar exercises and working around one missing idea), not that she wasn't trying. The common wrong reading is that a plateau means low effort.

7. A sample answer: "(Goal) I'm building a pivot table in Google Sheets that totals revenue by month. (Attempt) I selected A1:F200 and added Month as rows and Revenue as values, summarized by SUM. (Result) The March total is ₹48,200, but adding the March rows by hand gives ₹52,700. (Expectation) I expected the two to match. (Example) Here's a copy of the sheet with invented data, 12 rows, that shows the same difference." Writing this often reveals the cause, for example rows outside the selected range or revenue stored as text in some rows.

8. Sample outline. **Question:** which products sell best to hospitality customers, and should promotion change? **Data:** Riverstone one-year database (fictional), orders and products for 2025, hospitality segment only; note it excludes returns and tax. **Approach:** revenue and quantity by product for hospitality customers, compared with all customers; share of hospitality revenue per product. **Result:** one table of products ranked by hospitality revenue, with each product's share compared with its share overall. **Check:** hospitality product revenues add up to total hospitality revenue; one order hand-checked. **Decision:** promote the products where hospitality's share is high but sales are still small, and stop promoting products hospitality customers rarely buy. **Next:** repeat by quarter to see seasonality, and send it to the hospitality sales executive monthly. Any outline with all seven parts, a check, and a decision is acceptable; the analysis itself is built in Part II.

9. The log shows a **plateau**: scores rose from 4 to 6 by week 4, then stayed at 5–6 for six weeks while practice minutes kept rising. Likely causes: practice has become **comfortable** (repeating problems already mastered), or a **missing underlying idea** is blocking a family of problems; tool-hopping or

fatigue are also possible. Three changes: (1) review the mistakes list and look for a shared cause, then rebuild that section's examples without looking; (2) replace repeated exercises with new, harder problems, ideally the learner's own real questions; (3) get outside feedback, for example solving one problem aloud for a peer each week, and reduce minutes if fatigue is part of it.

10. Sample week (any design with specific goals, deliberate methods, and a feedback source is acceptable). **Monday:** goal: explain `PARTITION BY` versus `GROUP BY`; method: write a one-paragraph explanation from memory, then check against section 13.3; feedback: the chapter text. **Tuesday:** goal: predict the output of three window queries before running them; method: predict, then run; feedback: the real output. **Wednesday:** goal: rebuild the running-total example from its question alone; method: rebuild without looking; feedback: compare with the chapter's result. **Thursday:** goal: solve two new window problems on a different table; method: vary the problem; feedback: exercise answers. **Friday:** goal: explain one solution aloud to a peer or record it; method: explain it; feedback: the peer's questions, and a note of anything you couldn't explain.

11. The friend misreads the finding. The meta-analysis measured how much of the *differences between people* was explained by the *amount* of deliberate practice they reported; a small share in professions doesn't mean practice is unimportant. Everyone in those professions had already practiced a great deal, which shrinks the differences practice can explain, and on-the-job learning is hard to measure as "deliberate practice". The research suggests that practice matters and its quality matters most, and that in professional work other things also matter: working on real problems, getting real feedback, and learning from experienced people. That's why the three ingredients include projects and feedback, not practice alone.

12. Because early on, everything is new, so almost any practice is at the right difficulty and gives feedback. After a few months, you can already solve the familiar problems, so it's natural to drift toward practice that feels comfortable, and a single missing idea can block progress on the harder problems you now meet. Fatigue and competing commitments also build up over months.

13. Start a small project now, alongside the courses. Knowledge without projects doesn't turn into skill, and it's hard to know what "enough" is until a real problem shows you what's missing. With 8 hours a week, a split such as 5 hours of study and practice and 3 hours on a small project on their own domain gives them all three ingredients, and the project becomes a portfolio piece. The feeling of "not knowing enough" never fully goes away; it's a reason to build, not to wait.

14. A **mentor** can give judgment from experience: which skills matter in your company or city, which role fits you, how to handle a stakeholder, when your question is the wrong question, introductions to people, and encouragement from someone who has been through the same plateaus. An **AI assistant** can give instant, patient explanations at any hour, many variations of practice problems, and a quick first review of code, without using up anyone's time. The assistant can be confidently wrong and doesn't know your situation; the mentor's time is limited and their experience is one path. Use both, and check both.